КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики
Кафедра математичної інформатики

Кваліфікаційна робота
на здобуття ступеня магістра
за спеціальністю 122 Комп'ютерні науки
на тему:

ПІДВИЩЕННЯ ГЕНЕРАЛІЗАЦІЇ МОДЕЛІ РОЗПІЗНАВАННЯ ДАКТИЛЕМ УКРАЇНСЬКОЇ ЖЕСТОВОЇ МОВИ В УМОВАХ ОБМЕЖЕНИХ ДАНИХ

Виконав студент 2-го курсу
ОНП «Штучний інтелект»
Задорожний Михайло Олександрович _____
(підпис)

Науковий керівник:
член-кореспондент НАНУ, доктор фізико-математичних наук, професор
Крак Юрій Васильович _____
(підпис)

Засвідчую, що в цій роботі немає запозичень з праць інших авторів без відповідних посилань.
Студент _____
(підпис)

Роботу розглянуто й допущено до захисту
на засіданні кафедри математичної інформатики
«____» _____ 2026 р.,
протокол № ___
Завідувач кафедри
доктор фізико-математичних наук, професор
Терещенко Василь Миколайович _____
(підпис)

КИЇВ 2026

РЕФЕРАТ

Обсяг роботи: 59 сторінок, 13 ілюстрацій, 4 таблиці, 21 джерело посилань у тексті, 4 додатки.

Ключові слова:

РОЗПІЗНАВАННЯ ДАКТИЛЕМ УЖМ, ЗГОРТКОВІ НЕЙРОННІ МЕРЕЖІ, MEDIAPIPE, КЛЮЧОВІ ТОЧКИ, ГЕНЕРАЛІЗАЦІЯ.

Об’єктом дослідження є згорткові нейронні мережі та процес розв’язування задачі розпізнавання дактилем української жестової мови з їх використанням.

Предметом дослідження є особливості згорткових нейронних мереж, вплив способу тренування та модульності на генералізацію та використання на практиці.

Метою роботи є вирішення задачі розпізнавання дактилем української жестової мови, з нахилом на максимальну генералізацію, двома способами — одноетапною та багатоетапною класифікаціями, з подальшим порівнянням практичних результатів їх роботи.

Методи дослідження:

Вивчення численних публікацій фахівців, опрацювання запропонованої керівником літератури, пошук додаткової інформації із інших перевірених інтернет-джерел, проведення великої кількості експериментів.

Результати роботи:

Виконано тренування двох моделей розпізнавання дактилем української жестової мови, для котрого використано авторський набір даних[17]; Отримано дві моделі розпізнавання із практичними точностями top-1 F1-score рівними 91% для одноетапної та 86% для багатоетапної.

ЗМІСТ

РЕФЕРАТ 2
ЗМІСТ 3
СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ 4
ВСТУП 5
РОЗДІЛ 1. ЗАСТОСУВАННЯ ЗНМ ДЛЯ ВИРІШЕННЯ ЗАДАЧІ В ОДИН ЕТАП 8
1.1. Використана архітектура: глибокий опис лінійки X3D 8
1.2. Застосована вибірка 12
1.3. Застосовані аугментації даних 16
1.4. Рішення проблеми дизбалансу вибірки 17
1.5. Метод оптимізації та функція втрат 18
1.6. Керування швидкістю навчання 19
1.7. Додаткові налаштування тренування 20
РОЗДІЛ 2. ЗАСТОСУВАННЯ ДВОЕТАПНОГО СПОСОБУ ВИРІШЕННЯ ЗАДАЧІ ЗА УЧАСТЮ MEDIAPIPE 21
2.1. Детальніше про модель Mediapipe Hand Landmarker 21
2.2. Детальніше про архітектуру подальшої класифікації ключових точок 23
2.3. Застосована вибірка 25
2.4. Аугментації даних 29
2.5. Рішення проблеми дизбалансу вибірки 29
2.6. Оптимізатор, функція втрат, планувальник навчання 30
2.7. Додаткові налаштування 31
РОЗДІЛ 3. АНАЛІЗ РЕЗУЛЬТАТІВ ЕКСПЕРИМЕНТІВ 32
3.1. Результати тренування прямої ЗНМ для класифікації в один етап 32
3.2. Результати тренування ЗНМ для двоетапної класифікації через Mediapipe Hand Landmarker 36
ВИСНОВКИ 40
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ 44
ДОДАТОК А: код екстрактора кадрів із відео 46
ДОДАТОК Б: деталі архітектури для класифікації 50

ДОДАТОК В: код конвертації датасету в ключові точки..... 53
ДОДАТОК Г: код створених у рамках роботи програм тренування..... 59

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ

НМ — Нейронна мережа;

ЗНМ/CNN — Згорткова нейронна мережа, Convolutional Neural Network;

ROI — Region Of Interest, область інтересу;

УЖМ — Українська жестова мова.

ВСТУП

Актуальність та оцінка стану. Актуальність автоматизованого розпізнавання української жестової мови визначається необхідністю забезпечення рівноправного доступу до інформації та засобів комунікації для людей із порушеннями слуху, які щоденно зіштовхуються з труднощами під час взаємодії з чуючими людьми та цифровими системами. В Україні жестова мова є основним засобом спілкування для тисяч нечуючих громадян, і з 2022 року, на жаль, їхня кількість суттєво збільшується.

Перші дослідження розпізнавання жестів переважно використовували класичний комп'ютерний зір. Для української жестової мови застосовували вручну сформовані ознаки, зокрема гістограми орієнтованих градієнтів[1], bag-of-features[2] та розділення гіперплощинами[3]. Такі підходи забезпечували лише базову точність, були чутливими до освітлення й фону та погано працювали з великими обсягами даних.

Подальший розвиток став можливим завдяки нейронним мережам, особливо згортковим (CNN), які краще адаптуються до змін середовища. З появою 3D-CNN стало можливим аналізувати не окремі кадри, а часові послідовності, що відкрило шлях до розпізнавання динамічних жестів. Для багатьох жестових мов уже створено великі стандартизовані датасети (наприклад, WLASL[4], MS-ASL[5], How2Sign[6], AUTSL[7], CSL & SLR500[8]), тоді як для української жестової мови дослідження досі спираються переважно на невеликі локальні вибірки.

Наприклад, у [9] було створено датасет дактилем української жестової мови обсягом близько 50 тисяч кадрів, який після аугментації збільшили до 250 тисяч. На його основі сформували 50–80 тисяч коротких відео по 5 кадрів. Кількість відео безумовно суттєва, проте така довжина часто недостатня для жестів із рухом, тому подібні літери, як «Ґ» і «Г» або «Ї» та «І», важко надійно розрізняти. Додатковим недоліком стало використання модифікованої MobileNetV3[10], яку тренували з нуля:

модель із понад 3 млн параметрів є занадто складною для такого обмеженого набору відео.

Сфера застосування розпізнавання дактилем української жестової мови має широке практичне значення у сфері інклюзивних технологій та цифрової комунікації.

Насамперед такі системи можуть використовуватися для автоматичного перекладу жестового мовлення у текст або голос, що спрощує взаємодію між людьми з порушеннями слуху та тими, хто не володіє жестовою мовою. Це особливо важливо у державних установах, медичних закладах, освітньому середовищі та службах підтримки, де швидке й точне передавання інформації є критично важливим.

Окремим напрямом є застосування у навчальних системах. Програмні засоби з функцією розпізнавання дактилем можуть допомагати у вивченні української жестової мови, аналізуючи правильність виконання жестів у реальному часі та надаючи користувачеві зворотний зв’язок. Це дає змогу створювати інтерактивні платформи для самостійного навчання як для людей із порушеннями слуху, так і для перекладачів, викладачів або членів родини.

Ще однією сферою є людино-машинна взаємодія. Розпізнавання дактилем може використовуватися як альтернативний спосіб керування комп’ютерами, мобільними пристроями або спеціалізованими інформаційними системами без клавіатури чи голосових команд. Це відкриває можливості для створення як доступних інтерфейсів, адаптованих до потреб користувачів із різними формами інвалідності, так і просунутих технологій на кшталт наступних поколінь віртуальної реальності. У перспективі такі рішення можуть стати основою для розробки інтелектуальних помічників, здатних підтримувати повноцінний діалог українською жестовою мовою.

Підстави для виконання. Освітлені вище проблеми галузі розпізнавання УЖМ виявляють нестачу даних та нестабільність наявних рішень як суттєву прогалину, закриття котрої може стати поштовхом до розвитку технологій людино-машинної взаємодії.

Мета роботи:

- Дослідити особливості розв’язання задачі розпізнавання дактилем української жестової мови в умовах обмеженого обсягу навчальних даних та визначити підходи до підвищення здатності моделі до узагальнення;
- Провести навчання з опором на генералізацію за двома підходами до класифікації дактилем: одноетапний метод прямої класифікації послідовності зображень та багатоетапний метод із попереднім виділенням ключових точок руки;
- Проаналізувати вплив обох підходів побудови архітектури та навчання на якість розпізнавання, провести тестування та порівняння;
- Порівняти ефективність використання системи виділення ключових точок руки для подальшої класифікації жестів на основі просторово-часових координат відносно стандартного способу одноетапної просторово-часової класифікації.

Об’єктом дослідження є згорткові нейронні мережі та процес розв’язування задачі розпізнавання дактилем української жестової мови з використанням їх та комп’ютерного зору.

Предметом дослідження є особливості згорткових нейронних мереж, вплив способу тренування та модульності на генералізацію та використання на практиці.

Методами дослідження є вивчення численних публікацій фахівців, опрацювання запропонованої керівником літератури, пошук додаткової інформації із інших перевірених інтернет-джерел та проведення великої кількості експериментів.

Усі експерименти по тренуванню проводилися на базі Pytorch 2.10.0+cu130, torchvision 0.25.0+cu130, mediapipe 0.10.33 та сімейству суміжних бібліотек. Параметри застосованої системи:

CPU: Intel(R) Core(TM) i5-10500H CPU @ 2.50GHz

GPU: NVIDIA GeForce GTX 1650 with Max-Q Design

РОЗДІЛ 1. ЗАСТОСУВАННЯ ЗНМ ДЛЯ ВИРІШЕННЯ ЗАДАЧІ В ОДИН ЕТАП

1.1. Використана архітектура: глибокий опис лінійки X3D

Для досягнення поставлених цілей було використано архітектуру X3D-M[11], частину лінійки моделей X3D (Expand 3D), створених у рамках досліджень Facebook AI Research (FAIR) у 2020 році. Це сімейство моделей створене для задач результативного відеорозпізнавання та ґрунтується на концепції поетапного масштабування компактної початкової 2D-архітектури у просторово-часову (3D) структуру.

Ключова інноваційність підходу[11] полягає у відході від лінійної стратегії «інфляції», за якої ядра 2D-згорток прямо проєктуються у третій (часовий) вимір, як це реалізовано в традиційних архітектурах типу I3D[12]. Замість цього X3D застосовує поетапне масштабування вздовж різних вимірів архітектури (рис. 1). Під час конструювання моделі враховуються такі характеристики:

1) тривалість у часі (temporal duration) (γ_t),
2) частота дискретизації кадрів (frame rate) (γ_τ),
3) просторова деталізація (spatial resolution) (γ_s),
4) канальна ширина мережі (width) (γ_w),
5) ширина стискуючого (bottleneck) шару (γ_b),
6) глибина моделі, тобто число шарів (depth) (γ_d).

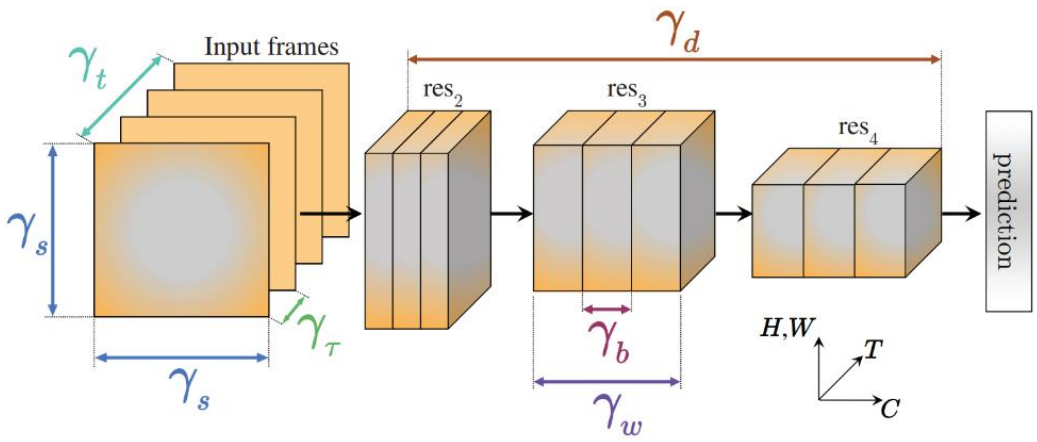


Рис. 1. Візуалізація прогресії розширення мереж X3D[11]

У рамках дослідження автори здійснювали покрокове розширення, збільшуючи значення одного із 6 параметрів: γ_t, γ_τ, γ_s, γ_w, γ_b, γ_d. Кращий із кроків визначався стратегією послідовних відборів — збільшується (у $\hat{c} = 2$ рази) кожен із 6 факторів, отримується та перевіряється 6 нових варіантів розширення, а найкращий вибирається по досягнутих показниках точності та розміру, стаючи при цьому основою для наступного кроку. Для кожної конфігурації автори задають цільове значення обчислювальної вартості c та ітеративно виконують кроки, доки його не буде досягнуто. Якщо на певному етапі найкращий варіант масштабування з коефіцієнтом $\hat{c} = 2$ призводить до завеликої обчислюваної вартості ($\gg c$) , коефіцієнт розширення $\hat{c}$ зменшують таким чином, щоб отримати кількість параметрів, наближену до $\approx c$. Детальніше вплив коефіцієнтів γ на структуру моделі проілюстровано на рис. 2.

Подібний підхід перегукується з класичним методом *forward feature selection* у машинному навчанні, де нові ознаки додаються поступово лише за умови їхнього відчутного внеску в якість. У випадку X3D це дає змогу підібрати архітектуру з раціональним компромісом між швидкодією та ефективністю.

stage	filters	output sizes $T \times S^2$
data layer	stride γ_τ, 1^2	$1\gamma_t \times (112\gamma_s)^2$
conv_1	1×3^2, 3×1, $24\gamma_w$	$1\gamma_t \times (56\gamma_s)^2$
res_2	$\begin{bmatrix} 1 \times 1^2, 24\gamma_b\gamma_w \\ 3 \times 3^2, 24\gamma_b\gamma_w \\ 1 \times 1^2, 24\gamma_w \end{bmatrix} \times \gamma_d$	$1\gamma_t \times (28\gamma_s)^2$
res_3	$\begin{bmatrix} 1 \times 1^2, 48\gamma_b\gamma_w \\ 3 \times 3^2, 48\gamma_b\gamma_w \\ 1 \times 1^2, 48\gamma_w \end{bmatrix} \times 2\gamma_d$	$1\gamma_t \times (14\gamma_s)^2$
res_4	$\begin{bmatrix} 1 \times 1^2, 96\gamma_b\gamma_w \\ 3 \times 3^2, 96\gamma_b\gamma_w \\ 1 \times 1^2, 96\gamma_w \end{bmatrix} \times 5\gamma_d$	$1\gamma_t \times (7\gamma_s)^2$
res_5	$\begin{bmatrix} 1 \times 1^2, 192\gamma_b\gamma_w \\ 3 \times 3^2, 192\gamma_b\gamma_w \\ 1 \times 1^2, 192\gamma_w \end{bmatrix} \times 3\gamma_d$	$1\gamma_t \times (4\gamma_s)^2$
conv_5	1×1^2, $192\gamma_b\gamma_w$	$1\gamma_t \times (4\gamma_s)^2$
pool_5	$1\gamma_t \times (4\gamma_s)^2$	$1 \times 1 \times 1$
fc_1	1×1^2, 2048	$1 \times 1 \times 1$
fc_2	1×1^2, #classes	$1 \times 1 \times 1$

Рис. 2. Узагальнене представлення базової моделі (X2D) та процесу її масштабування до архітектур X3D [11]. Параметри згорткових ядер задаються у форматі $\{T \times S^2, C\}$, де T — часовий розмір, S^2-просторовий розмір, C — кількість каналів. Значення кроків (strides) подаються як {часовий крок, просторовий крок^2}.

Серед різних екземплярів (X3D-XS, X3D-S, X3D-M, X3D-L, X3D-XL, X3D-XXL) було вирішено застосувати X3D-M, баланс точності та обчислюваної складності котрої (рис. 3) найкраще підходив для тренування у рамках даної роботи. Ключові властивості моделі X3D-M охоплюють:

- параметри масштабування: $\gamma_t = 16, \gamma_\tau = 5,\ \gamma_s = 2,\ \gamma_w = 1,\ \gamma_b = 2.25,\ \gamma_d = 2.2$;
- обчислювальні витрати: орієнтовно 4.73 GFLOPs на один відеофрагмент і близько 3.76 млн параметрів;
- досягнуті показники якості: точність становить 74.6% за метрикою Top-1 та 91.7% за Top-5 на наборі Kinetics-400[13];
- вхідні характеристики: відеопослідовності довжиною $\gamma_t = 16$ кадрів із просторовою роздільністю $(112 * \gamma_s,\ 112 * \gamma_s) = (224,\ 224)$.

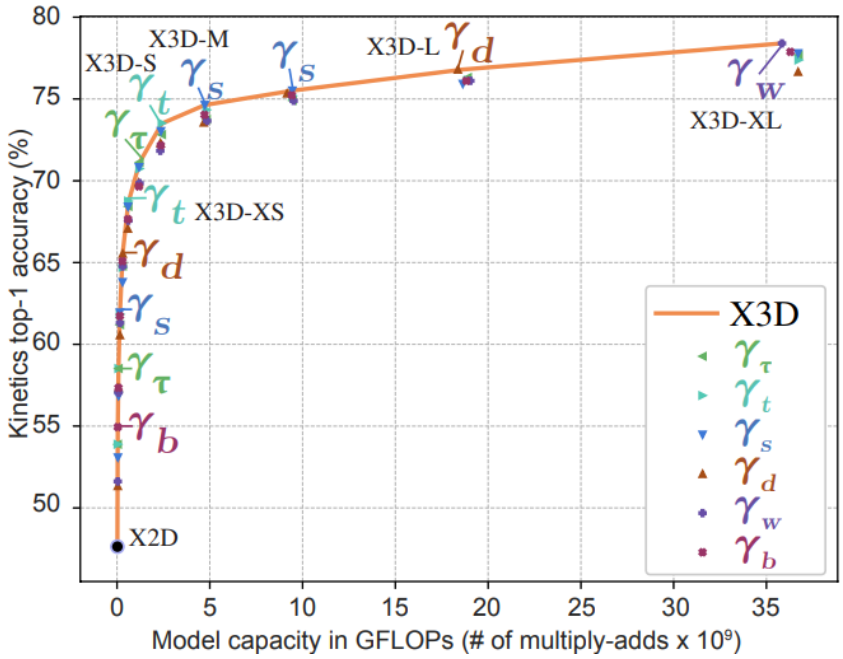


Рис. 3. Порівняльний графік результатів моделі X3D-M[11] відносно інших членів лінійки Expand 3D на наборі Kinetics-400[13]. Для кожного з 11 етапів наведено показники всіх доступних варіантів масштабування (згідно з легендою); найефективніші рішення на кожному кроці виділені окремо.

Усі представники сімейства X3D, зокрема й X3D-M, тренувалися з нульовою ініціалізацією безпосередньо на наборі даних для розпізнавання дій Kinetics-400[13], без попереднього навчання на ImageNet. Набір Kinetics-400 включає приблизно 240 тис. тренувальних, 20 тис. валідаційних і 35 тис. тестових відеозаписів, що охоплюють 400 категорій людської активності. Використання виключно відеоданих обумовлене прагненням уникнути зміщень, притаманних навчанню на статичних зображеннях, і забезпечити більш «чистий» результат, орієнтований саме на аналіз відео.

Крім того, для оцінки здатності до узагальнення запропоновану архітектуру перевіряли на наборах Kinetics-600[14], Charades[15] та AVA[16]. У всіх експериментах модель показувала високі результати: в окремих випадках вона наближалася до рівня state-of-the-art рішень (до появи підходів на основі ViT, які досягають вищої якості ціною кратного зростання кількості параметрів), водночас потребуючи приблизно у 4.8× менше FLOPs та у 5.5× менше параметрів[11].

Отже, модель X3D-M, попередньо навчена на Kinetics-400, може розглядатися як доцільна базова ініціалізація для задачі розпізнавання дактилем української жестової мови.

1.2. Застосована вибірка

Для тренування й тестування моделі розпізнавання української дактильної абетки було створено власний відео-сет[17], що налічує приблизно 3000 кліпів. Під час формування вибірки особливу увагу приділено різноманітності умов зйомки: освітленню, задньому фону, одягу учасників та індивідуальним особливостям кистей рук.

Вибірка структурована за трьома функціональними підмножинами:

- *Навчальна частина (директорія Train):* містить 1951 відеозапис. Цей піднабір є незбалансованим — кількість прикладів для окремих класів варіюється в межах від 25 до 72 кліпів.
- *Валідаційна частина (директорія Val):* включає 528 відео. Вона повністю збалансована — по 16 прикладів на кожну з 33 літер. Кожен клас представлений 8 оригінальними записами та 8 їхніми варіантами, отриманими шляхом горизонтального віддзеркалення.
- *Тестова частина (директорія Test):* має аналогічну структуру до валідаційної — 528 відео (по 16 на клас: 8 початкових і 8 дзеркально трансформованих).

Розподіл на ці підмножини виконувався на рівні повних відеофайлів. Жодні кадри чи фрагменти з одного джерельного відео не перетинаються між різними частинами вибірки, що виключає витік інформації та гарантує коректну перевірку здатності моделі до узагальнення.

Кожен відеозапис представлено як послідовність із 16 RGB-кадрів; при цьому роздільна здатність кліпів може відрізнятися. Незбалансованість навчальної частини

зумовлена об’єднанням даних із кількох локальних джерел запису. Окрім підготовленої вибірки кадрів (надалі — «вибірка»), у [17] також надано вихідні відеофайли, впорядковані за місцями зйомки (по одному відео на клас), що дає змогу формувати послідовності з довільною кількістю кадрів, відмінною від 16. Більше того, застосований у роботі екстрактор кадрів, тобто його код, прикріплено у додатку А.

Важливо відзначити, що навчальна підмножина містить відеозаписи лише правої руки, оскільки різноманітність для лівої компенсується шляхом програмного дзеркального перетворення на етапі аугментації. Крім того, цільові жести у цих записах розташовані переважно поблизу центру кадру з метою підвищення ефективності застосовуваних аугментацій. Для наочнішого представлення дисбалансу навчальної вибірки було побудовано гістограму (рис. 4).

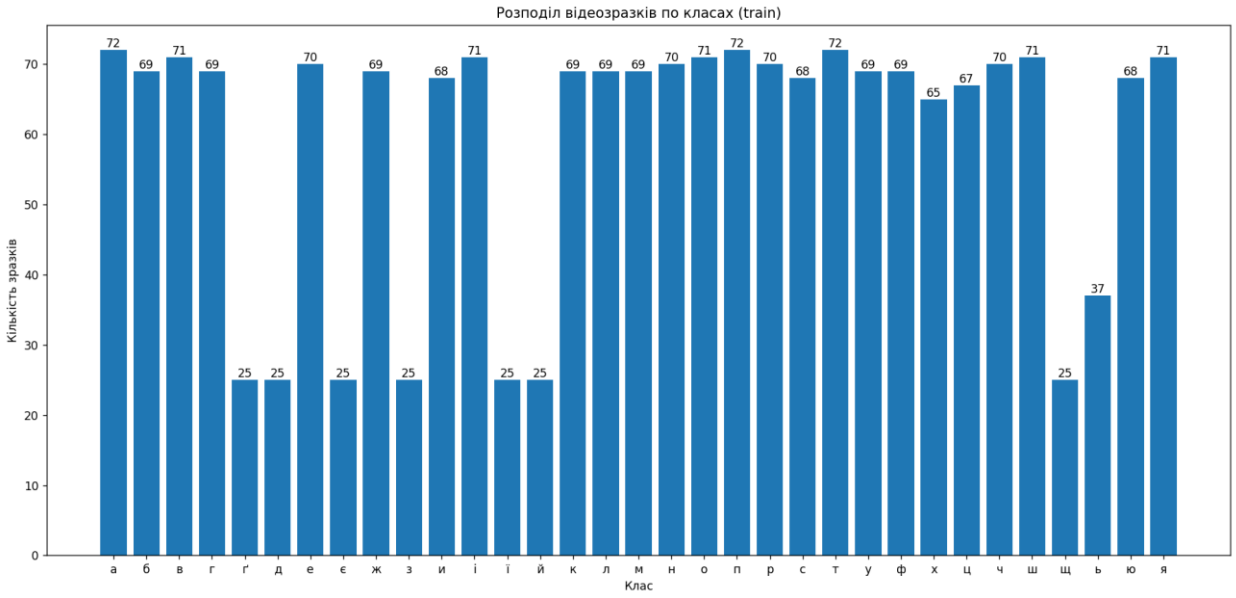


Рис. 4: Гістограма зразків тренувальної вибірки

Окремі складові зібраного набору містили інформацію лише для 26 із 33 категорій, тобто охоплювали виключно «статичні» жести. Незважаючи на це, їх було включено до вибірки з метою істотного розширення варіативності даних.

Навчальну, валідаційну та тестову підмножини наведено на рисунках 5–7 відповідно.



Рис. 5: Ілюстрація тренувальної вибірки, фрейми



Рис. 6: Ілюстрація валідаційної вибірки, фрейми



Рис. 7: Ілюстрація тестової вибірки, фрейми

1.3. Застосовані аугментації даних

З метою підвищення різноманітності вибірки до кожного відео покадрово застосовується набір онлайн-аугментацій, що виконуються послідовно:

- випадкове обрізання з масштабуванням;
    - Масштабування варіюється в межах 75–100% від початкового розміру, співвідношення сторін — у діапазоні від 0.875 (7/8) до 1.14 (8/7), із приведенням до фінальної роздільності: 224×224.
- випадкові корекції яскравості, контрастності, насиченості та тону;
    - яскравість, контраст і насиченість змінюються в межах ± 20%, тоді як відтінок — у межах ± 5%.
- віддзеркалення по горизонталі з імовірністю виконання 50%;
- обертання за випадковим напрямом;
    - кут повороту обмежено значенням до15 градусів;
    - специфіка: перед обертанням застосовується розширення меж зображення (edge-padding) на 20% від його розміру з подальшим центральним вирізанням, що дозволяє уникнути появи порожніх областей;
- додавання Ґаусівського шуму та розмиття.
    - для шуму використовується нормальний розподіл із середнім 0 та стандартним відхиленням $\sigma = 0.08$;
    - для розмиття застосовується Ґаусів фільтр із ядром розміру 5 та випадковим значенням σ в діапазоні від 0.1 до 2.0.

Візуальні приклади отриманих трансформацій наведено на рис. 8.

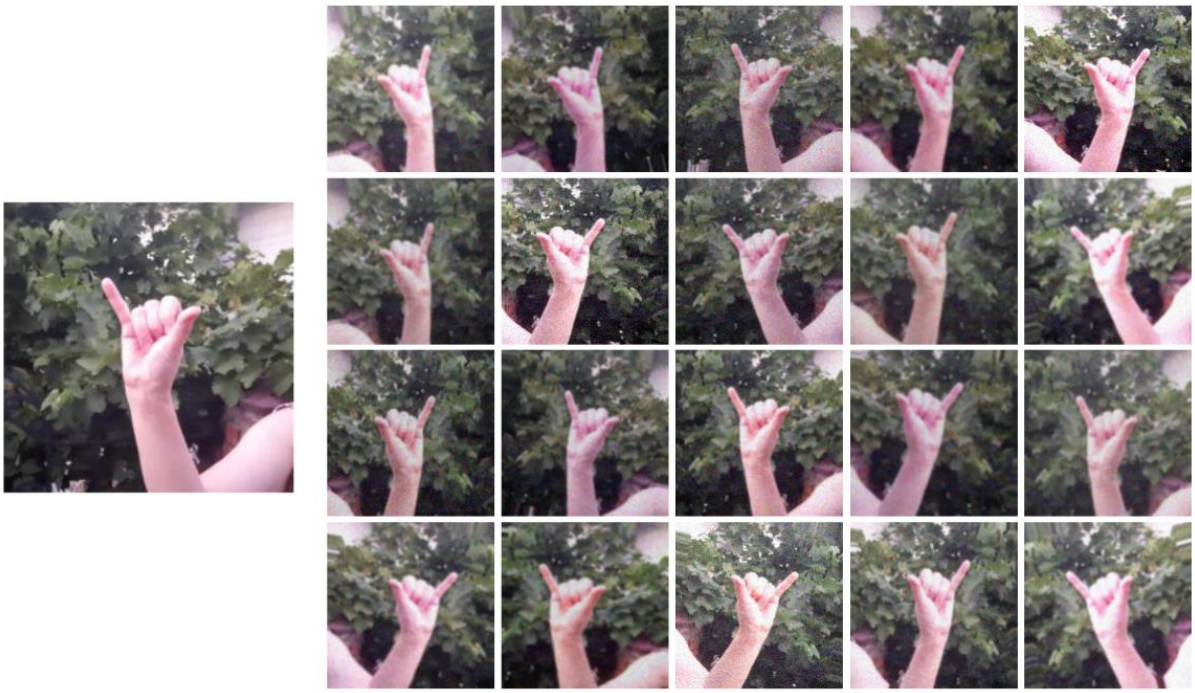


Рис. 8: Приклад візуального представлення результатів аугментацій для першого кадру відео: ліворуч наведено початкове зображення, праворуч — 20 варіантів, отриманих після застосування перетворень

1.4. Рішення проблеми дизбалансу вибірки

Щоб запобігти упередженню моделі в бік більш представлених класів, під час навчання використовується імовірнісне формування пакетів (batch) із вагами, обернено пропорційними частотам класів:

$$w_c{}^0 = \frac{1}{N_c}, \tag{1}$$

Де $w_c{}^0$ — початковий коефіцієнт ваги для класу c; N_c — кількість зразків класу c.

Застосування такого підходу дозволяє досягти статистично рівномірного представлення всіх категорій упродовж кожної епохи тренування.

У разі потреби, зокрема при потраплянні моделі в локальні мінімуми, базові «балансуючі» ваги додатково коригуються на основі аналізу помилок класифікації (відповідна статистика зберігається у файлах). Для цього використовується

логарифмічно обернене масштабування точності по діагональних елементах матриці невідповідностей:

$$w_c = {w_c}^0 * \left(\ln\left(\frac{1}{d_c + \epsilon}\right) + 1\right), \quad (2)$$

Де w_c — підсумковий ваговий коефіцієнт для класу c; d_c — точність класифікації класу c, ϵ — мале додатне число для уникнення сингулярності.

Значення ϵ було встановлено на 0.03 для обмеження $\ln(\dots)$ до 3.5 при $d_c \to 0$, тоді як додавання одиниці у формулі не допускає повного виключення класу з вибірки при $d_c \to 1$. Така модифікація дозволяє підвищити ймовірність включення прикладів із класів, які модель розпізнає гірше, аж до чотириразового збільшення.

1.5. Метод оптимізації та функція втрат

Як функцію втрат було обрано перехресну ентропію. Для оптимізації параметрів застосовано алгоритм Adam[18], який динамічно підлаштовує крок навчання окремо для кожного параметра, беручи до уваги як середнє значення градієнтів, так і їх варіацію. Це сприяє швидшій збіжності та робить процес тренування більш стабільним у випадках значної різноманітності даних.

Водночас відомим недоліком Adam є обмежена сумісність із регуляризаціями типу L1 та L2, оскільки вони включаються до складу градієнтів і тим самим впливають на оцінки моментів. З цієї причини у роботі [19] запропоновано альтернативний підхід: винесення регуляризаційного члена за межі градієнта із введенням коефіцієнта згасання ваг λ та корекцією параметрів шляхом віднімання $\lambda * \theta_t$ під час кожного оновлення до θ_{t+1}. Така модифікація отримала назву AdamW [19] і саме вона була використана в експериментах. Подібна схема також сприяє швидшій збіжності та підвищує стабільність навчання на варіативних даних.

Параметри налаштування обраного оптимізатора мають такий вигляд:

- Базове значення швидкості навчання встановлено на рівні 0.001,

- коефіцієнти експоненційного згладжування для першого та другого моментів залишено стандартними, як у Adam[18]: $\beta_1 = 0.9,\ \beta_2 = 0.999$;
- параметр «чутливості» оптимізатора задано як $\epsilon = 10^{-8}$;
- коефіцієнт згасання ваг змінюється протягом навчання: від $\lambda_{start} = 10^{-3}$ на початковому етапі до $\lambda_{mid} = 10^{-2}$ у середині процесу та знову до $\lambda_{end} = 10^{-3}$ наприкінці.

1.6. Керування швидкістю навчання

У зв'язку зі складною багатовимірною поверхнею функції втрат, що містить велику кількість локальних мінімумів, окрім використання ефективного оптимізатора було додатково впроваджено механізм керування швидкістю навчання. В основі планувальника лежить підхід косинусного спадання з рестартами (cosine annealing with warm restarts), запропонований у [20].

На відміну від класичної схеми, де після кожного рестарту швидкість навчання повертається до початкового максимального значення, застосовано модифікований варіант із мультиплікативним масштабуванням. У цьому випадку після завершення кожного циклу відбувається перезапуск із поступовим зменшенням амплітуди за рахунок введення коефіцієнта згасання (decay_factor). Такий підхід дає змогу на початкових етапах ширше досліджувати простір параметрів, а на пізніших — звужувати область пошуку, зокрема завдяки можливості адаптивної зміни цього коефіцієнта.

У результаті експериментального підбору було визначено такі налаштування планувальника:

$$warmup = 0;\ T_0 = 10;\ T_{mult} = 1.25;\ decay_factor = 0.9$$

де $warmup$ — кількість стартових епох, протягом яких швидкість навчання лінійно зростає від нуля до базового рівня; T_0 — початкова тривалість одного циклу (у

епохах); T_{mult} — коефіцієнт збільшення довжини кожного наступного циклу; $decay_factor$ — коефіцієнт згасання максимальної амплітуди за цикл.

1.7. Додаткові налаштування тренування

Для розрахунку градієнтів використовується ефективний розмір пакета 64, який через обмеження пам’яті реалізується як послідовна обробка мініпакетів по 4 відео із подальшим накопиченням градієнтів та відповідним коригуванням оновлень.

Такий підхід має помітний недолік — підвищену нестабільність і часті коливання буферних статистик у шарах batch normalization. Можливість використання альтернативних варіантів нормалізації розглядалася, проте від неї відмовилися через відсутність узгоджених статистик, отриманих під час попереднього навчання на Kinetics-400. Водночас, завдяки наявності цієї ініціалізації, зменшення коефіцієнта моменту для шарів нормалізації виявилося достатнім для стабілізації процесу навчання.

Параметр dropout у моделі X3D залишено на рівні, рекомендованому авторами архітектури, із ймовірністю p = 0.5. Очікується, що збіжність моделі буде досягнута приблизно за 170 епох.

РОЗДІЛ 2. ЗАСТОСУВАННЯ ДВОЕТАПНОГО СПОСОБУ ВИРІШЕННЯ ЗАДАЧІ ЗА УЧАСТЮ MEDIAPIPE

Іншим варіантом вирішення задачі розпізнавання дактилем УЖМ в умовах обмеженої кількості даних є використання open-source моделі, котра розпізнає ключові точки руки — Mediapipe Hand Landmarker[21]. Тобто, задача перетворюється із прямої класифікації картинок з проблемами масштабу, фонів, тіней, положень — на задачу класифікації за 21 точками руки.

2.1. Детальніше про модель Mediapipe Hand Landmarker

Модель MediaPipe Hand Landmarker є складовою фреймворку MediaPipe, розробленого компанією Google для побудови високопродуктивних систем комп'ютерного зору в реальному часі. Вона призначена для детекції та трекінгу кисті руки, а також для оцінки просторового положення її ключових анатомічних точок (landmarks) на зображеннях або відеопотоці. Ця модель широко використовується в задачах жестового управління, доповненої реальності, біомеханічного аналізу та людино-машинної взаємодії.

Архітектурно Hand Landmarker базується на дворівневому підході, який поєднує детектор долоні та модель регресії ключових точок. На першому етапі використовується легковагова згорткова нейронна мережа, що виконує детекцію області долоні у вхідному зображенні. Цей компонент оптимізований для високої швидкодії та здатний стабільно знаходити руки навіть за складних умов освітлення, часткового перекриття або різних орієнтацій. Після визначення області інтересу (ROI) відбувається її нормалізація, що дозволяє зменшити вплив масштабування та повороту.

На другому етапі застосовується окрема нейронна мережа, яка виконує регресію координат 21 ключової точки кисті. Ці точки відповідають анатомічно значущим позиціям, таким як суглоби пальців і центр долоні. Модель повертає як двовимірні координати у площині зображення, так і тривимірні координати у нормалізованому

просторі, де вісь глибини оцінюється відносно положення камери. Такий підхід дозволяє не лише визначати форму руки, а й оцінювати її просторову конфігурацію.

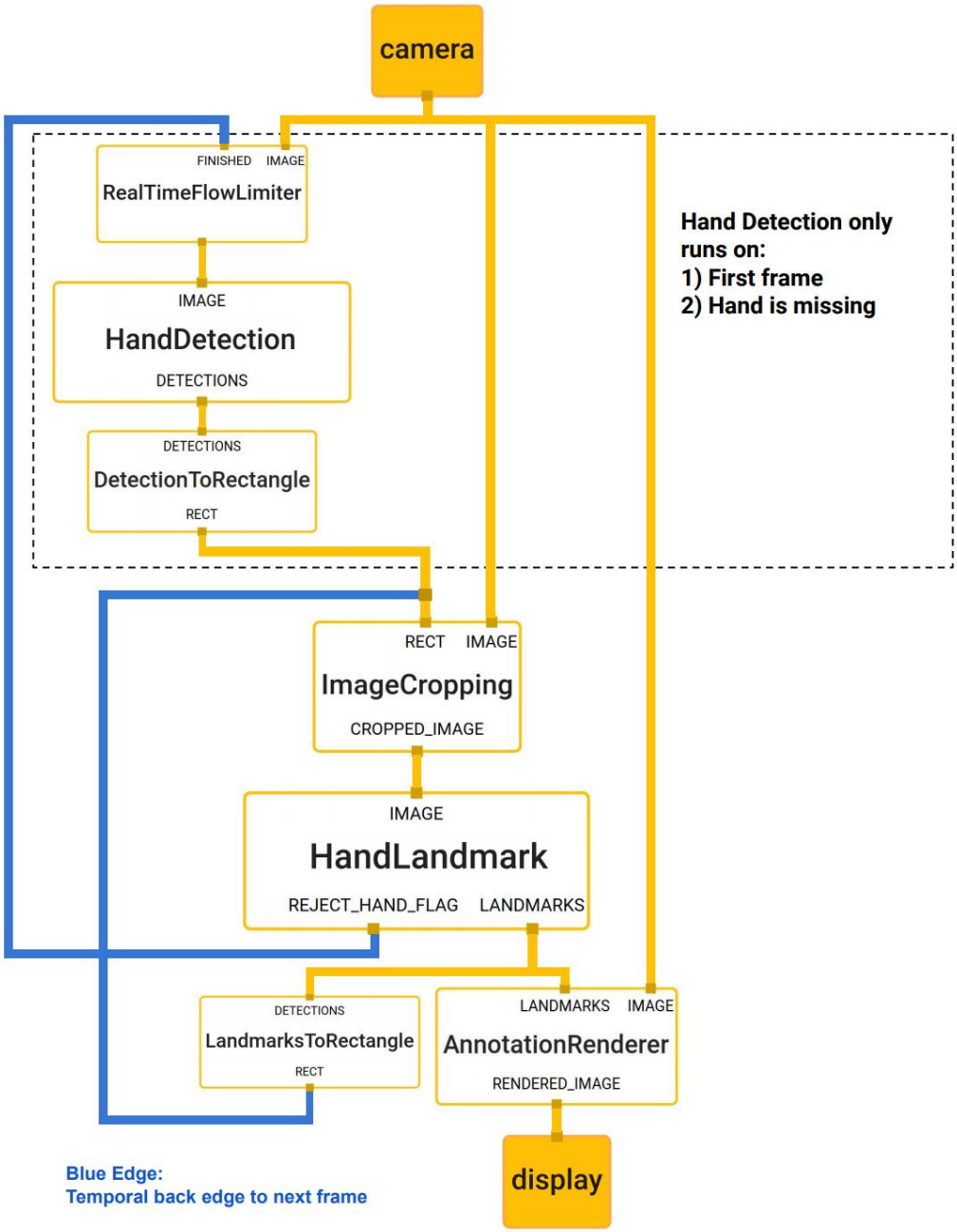


Рис. 9: базова візуалізація роботи Mediapipe Hand Landmarker[21]

Важливою характеристикою моделі є її здатність до трекінгу у відеопотоці. Після початкової детекції руки модель може використовувати інформацію з попередніх кадрів для прогнозування положення кисті в наступних, що значно знижує обчислювальне навантаження та підвищує стабільність роботи. У випадку втрати

трекінгу повторно активується детектор долоні. Така стратегія забезпечує баланс між точністю та продуктивністю.

Модель оптимізована для роботи на різних апаратних платформах, включаючи мобільні пристрої, CPU та GPU, що досягається завдяки використанню легковагових архітектур і квантизації. Вона також інтегрується з іншими компонентами MediaPipe, що дозволяє створювати складні мультимодальні системи.

З точки зору навчання, модель була натренована на великому наборі даних, що включає зображення рук у різних позах, масштабах і умовах освітлення. Для підвищення узагальнювальної здатності застосовувалися методи аугментації даних, зокрема випадкові обертання, масштабування та зміни яскравості. Крім того, для отримання тривимірних координат використовувалися як реальні, так і синтетично згенеровані дані.

Таким чином, MediaPipe Hand Landmarker є ефективним та універсальним рішенням для задач трекінгу рук у реальному часі, що поєднує високу точність локалізації ключових точок із низькими вимогами до обчислювальних ресурсів, що робить його придатним для широкого спектра прикладних досліджень і практичних застосувань.

2.2. Детальніше про архітектуру подальшої класифікації ключових точок

Запропонована модель класифікації жестів побудована для обробки часових послідовностей ключових точок кисті, отриманих за допомогою MediaPipe Hand Landmarker, і реалізована з використанням бібліотеки PyTorch. Її архітектура орієнтована на виділення як просторових, так і динамічних характеристик руху руки, що показало гарний ефект при роботі з відеопослідовностями фіксованої довжини.

На вході модель приймає тензор розмірності (B, 16, 42), де 16 — кількість часових кроків (кадрів), а 42 — координати 21 ключової точки кисті у двовимірному просторі (x, y). Так, Hand Landmarker повертає іще третю координату ключових точок z та розміри у метричній системі, але в контексті роботи з картинкою ці параметри

прогнозування міститимуть зависокий рівень шуму, при цьому не даючи великої кількості корисної інформації.

Початковим етапом обробки є нормалізація координат, яка реалізована у вигляді окремого модуля. У цьому блоці здійснюється приведення координат до відносної системи відліку, де всі точки виражаються відносно зап’ястя, що дозволяє досягти інваріантності до глобального положення руки в кадрі. Додатково виконується нормалізація за масштабом шляхом ділення координат на відстань між зап’ястям та одним із базових суглобів середнього пальця, що забезпечує інваріантність до розміру руки та відстані до камери. У режимі навчання до нормалізованих даних додається невеликий випадковий шум, який виконує роль регуляризації та підвищує стійкість моделі до незначних варіацій у вхідних даних.

Після нормалізації формується додаткове представлення у вигляді швидкостей руху, яке обчислюється як різниця координат між сусідніми кадрами. Таким чином модель отримує два паралельних потоки даних: статичні координати та їх часові зміни, що дозволяє враховувати як форму жесту, так і його динаміку. Обидва ці потоки обробляються незалежними, але ідентичними за структурою енкодерами.

Кожен енкодер є згортковою нейронною мережею, що працює вздовж часової осі та використовує одномірні згортки. На початковому етапі виконується проєкція вхідних ознак у простір більшої розмірності, що дозволяє моделі сформувати багатовимірне представлення сигналу. Далі застосовуються резидуальні блоки, які реалізують ідею залишкових з’єднань, подібно до архітектур сімейства ResNet. Це дозволяє покращити проходження градієнтів під час навчання та підвищити стабільність оптимізації. Один із блоків виконує двократне зменшення часової роздільної здатності, що сприяє виділенню більш абстрактних часових залежностей і зменшенню обчислювальної складності.

У фінальній частині енкодера застосовується адаптивне агрегування середнім вздовж часової осі, котре дозволяє агрегувати інформацію з усієї послідовності у

фіксований вектор ознак і водночас не сильно залежати від викидів та артефактів, котрі можуть виникнути як результат роботи Mediapipe. Також це дозволяє моделі працювати з часовими залежностями без використання рекурентних механізмів, зберігаючи при цьому ефективність обчислень. Отримані з енкодерів вектори ознак для координат і швидкостей конкатенуються, формуючи узагальнене представлення жесту, яке містить як просторову, так і динамічну інформацію.

Завершальний етап моделі представлений повнозв'язним класифікатором, що виконує відображення об'єднаного вектора ознак у простір класів жестів. У цьому блоці використовується нормалізація, нелінійність та Dropout, що знижує ризик перенавчання та покращує узагальнювальну здатність моделі. Вихідним результатом є логіти класів, які можуть бути перетворені у ймовірності за допомогою softmax-функції.

У цілому, архітектура моделі характеризується чітким розділенням обробки статичних і динамічних ознак, використанням інваріантних до положення і масштабу представлень, а також ефективним застосуванням згорткових механізмів для моделювання часових залежностей. Це робить її придатною для задач розпізнавання жестів у реальному часі, особливо в поєднанні з системами детекції ключових точок, такими як MediaPipe. Глибокі деталі архітектури наведено у додатку Б.

2.3. Застосована вибірка

Для навчання та оцінки моделі класифікації ключових точок тренувальний та валідаційний сети набору відеоданих[17] було сконвертовано шляхом запускання Mediapipe Hand Landmarker на кожному зі зразків вибірки. При цьому деякі із відео мали надто складну комбінацію фонів та тіней, котрі модель Mediapipe не змогла зчитувати та повертала відсутність руки у якості результату. Для великої кількості відео, де кількість нерозпізнаних фреймів була менше 4 із 16, було застосовано покоординатну кубічну (крайові значення повторювались) інтерполяцію, котра гарно зберегла послідовність рухів у часі та дозволила зменшити кількість втрачених відео.

1) ***тренуальний набір:*** інтерполяція зменшила кількість втраченого відео із 12.9% до 3.2%. Детальні статистики конвертації по кожному із класів було зібрано у таблиці …

Таблиця 1: Результати перетворення тренувального сету із картинок у ключові точки

Клас	**Всього відео**	**Інтерпольовано**	**Відкинуто**	**інтерпольовано, %**	**відкинуто, %**
сумарно	***1951***	***189***	***63***	***9.687***	***3.229***
а	72	3	3	4.167	4.167
б	69	6	1	8.696	1.449
в	71	10	3	14.085	4.225
г	69	10	2	14.493	2.899
ґ	25	1	1	4	4
д	25	18	1	72	4
е	70	6	2	8.571	2.857
є	25	0	0	0	0
ж	69	7	1	10.145	1.449
з	25	10	2	40	8
и	68	4	0	5.882	0
і	71	3	1	4.225	1.408
ї	25	4	2	16	8
й	25	6	0	24	0
к	69	2	6	2.899	8.696
л	69	5	4	7.246	5.797
м	69	6	0	8.696	0
н	70	3	0	4.286	0
о	71	5	1	7.042	1.408

п	72	8	2	11.111	2.778
р	70	6	0	8.571	0
с	68	8	1	11.765	1.471
т	72	11	1	15.278	1.389
у	69	5	0	7.246	0
ф	69	11	8	15.942	11.594
х	65	6	5	9.231	7.692
ц	67	5	1	7.463	1.493
ч	70	4	1	5.714	1.429
ш	71	5	3	7.042	4.225
щ	25	0	5	0	20
ь	37	1	1	2.703	2.703
ю	69	5	3	7.246	4.348
я	70	5	2	7.143	2.857

2) ***валідаційний набір:*** інтерполяція зменшила кількість втраченого відео із 9.3% до 3.4%. Детальні статистики конвертації по кожному із класів було зібрано у таблиці 3

Таблиця 2. Результати перетворення валідаційного сету із картинок у ключові точки

Клас	**Всього відео**	**Інтерпольовано**	**Відкинуто**	**інтерпольовано, %**	**відкинуто, %**
сумарно	***528***	***31***	***18***	***5.871***	***3.409***
а	16	2	0	12.5	0
б	16	0	0	0	0
в	16	0	0	0	0
г	16	0	0	0	0
д	16	11	0	68.75	0
е	16	0	3	0	18.75

ж	16	2	0	12.5	0
з	16	1	0	6.25	0
и	16	0	0	0	0
й	16	0	0	0	0
к	16	2	0	12.5	0
л	16	1	0	6.25	0
м	16	1	3	6.25	18.75
н	16	0	0	0	0
о	16	0	0	0	0
п	16	1	0	6.25	0
р	16	0	0	0	0
с	16	0	0	0	0
т	16	0	1	0	6.25
у	16	0	2	0	12.5
ф	16	0	5	0	31.25
х	16	0	0	0	0
ц	16	0	0	0	0
ч	16	2	0	12.5	0
ш	16	2	0	12.5	0
щ	16	0	0	0	0
ь	16	0	2	0	12.5
ю	16	0	0	0	0
я	16	0	2	0	12.5
є	16	0	0	0	0
і	16	2	0	12.5	0
ї	16	0	0	0	0
ґ	16	4	0	25	0

3) *Тестовий набір:* було залишено без змін. Прохід по тестовому набору відбувався за повною участю обох моделей — як Mediapipe Hand Landmarker, так і класифікатора ключових точок.

Код екстрактора ключових точок наведено у додатку В.

2.4. Аугментації даних

Для моделі класифікації ключових точок відео організовано як послідовності по T=16 кадрів, кожен описаний 42-ма координатами, тобто 21 ключова точка руки, (x, y) для кожної. Для збільшення варіативності даних до точок покадрово застосовується послідовність онлайн-аугментацій:

- випадкове віддзеркалення по горизонталі;
    - Ймовірність горизонтального перевороту: 50%
- обертання у випадкову сторону;
    - Діапазон: до 15 градусів.
- Випадкове розтягування руки по ширині чи по довжині;
    - У якості осі y для перетворень береться пряма між точкою зап'ястя (точка 0) та основою середнього пальця (точка 9) — ця пряма найкраще відповідає центру руки та забезпечує природнє перетворення. Вісь x, відповідно, перпендикулярна;
    - Параметри масштабу — від 0.8 до 1.2 по x та по y.
- Випадкове тремтіння координат (+- 4% відносно значення).

Усі аугментації проводяться до нормалізації.

2.5. Рішення проблеми дизбалансу вибірки

Для уникнення зміщення моделі у бік розповсюдженіших класів при тренуваннях, аналогічно пункту 1.4, застосовано імовірносний відбір пакетів (batch) на основі обернених частоті класів вагових коефіцієнтів (формула 1).

Відносно попереднього експерименту, для цього теж використовується логарифмічне обернене зважування точності по діагоналі матриці невідповідностей (формула 2) , але застосовуються автоматично після кожних *k=3* епох починаючи з епохи *n=50*. ϵ, тобто мале додатне число для уникнення сингулярності, було встановлено на *0.25* для обмеження *ln(...)* до *2.3* при $d_c \to 0$.

2.6. Оптимізатор, функція втрат, планувальник навчання

Аналогічно попередньому експерименту, перехресна ентропія показала себе гарним критерієм вирішення цієї задачі. Для навчання моделі класифікації використовувався оптимізатор AdamW[19].

Параметри налаштування обраного оптимізатора мають такий вигляд:

- Базове значення швидкості навчання встановлено на рівні 0.001,
- коефіцієнти експоненційного згладжування для першого та другого моментів залишено стандартними, як у Adam[18]: $\beta_1 = 0.9,\ \beta_2 = 0.999$;
- параметр «чутливості» оптимізатора задано як $\epsilon = 10^{-8}$;
- коефіцієнт згасання ваг змінюється протягом навчання: від $\lambda_{start} = 10^{-3}$ на початковому етапі до $\lambda_{mid} = 10^{-2}$ у середині процесу та знову до $\lambda_{end} = 10^{-3}$ наприкінці.

Планувальник вибрано аналогічний попередньому експерименту, оскільки він показав гарний потенціал, демонструючи стабільність результату ціною збільшення фактичного часу тренування. У пошуках компромісу було визначено такі параметри планувальника:

$$warmup = 4; T_0 = 10;\ T_{mult} = 1.2;\ decay_factor = 0.8$$

де $warmup$ — кількість стартових епох, протягом яких швидкість навчання лінійно зростає від нуля до базового рівня; T_0 — початкова тривалість одного циклу (у епохах); T_{mult} — коефіцієнт збільшення довжини кожного наступного циклу; $decay_factor$ — коефіцієнт згасання максимальної амплітуди за цикл.

2.7. Додаткові налаштування

Для обчислення градієнтів використовується розмір пакету = 64, без емуляцій — модель відносно легка, кількість параметрів становить приблизно 350 тисяч.

Dropout для архітектури класифікації ключових точок було встановлено на $p = 0.4$. Кількість епох до тренування становила 90, але найкращий результат був отриманий на 64 епосі.

РОЗДІЛ 3. АНАЛІЗ РЕЗУЛЬТАТІВ ЕКСПЕРИМЕНТІВ

3.1. Результати тренування прямої ЗНМ для класифікації в один етап

Для оцінювання ефективності моделі було побудовано матрицю невідповідностей (рис. 9) та обчислено покласові значення метрик precision і recall (табл. 1) на тестовій підмножині:

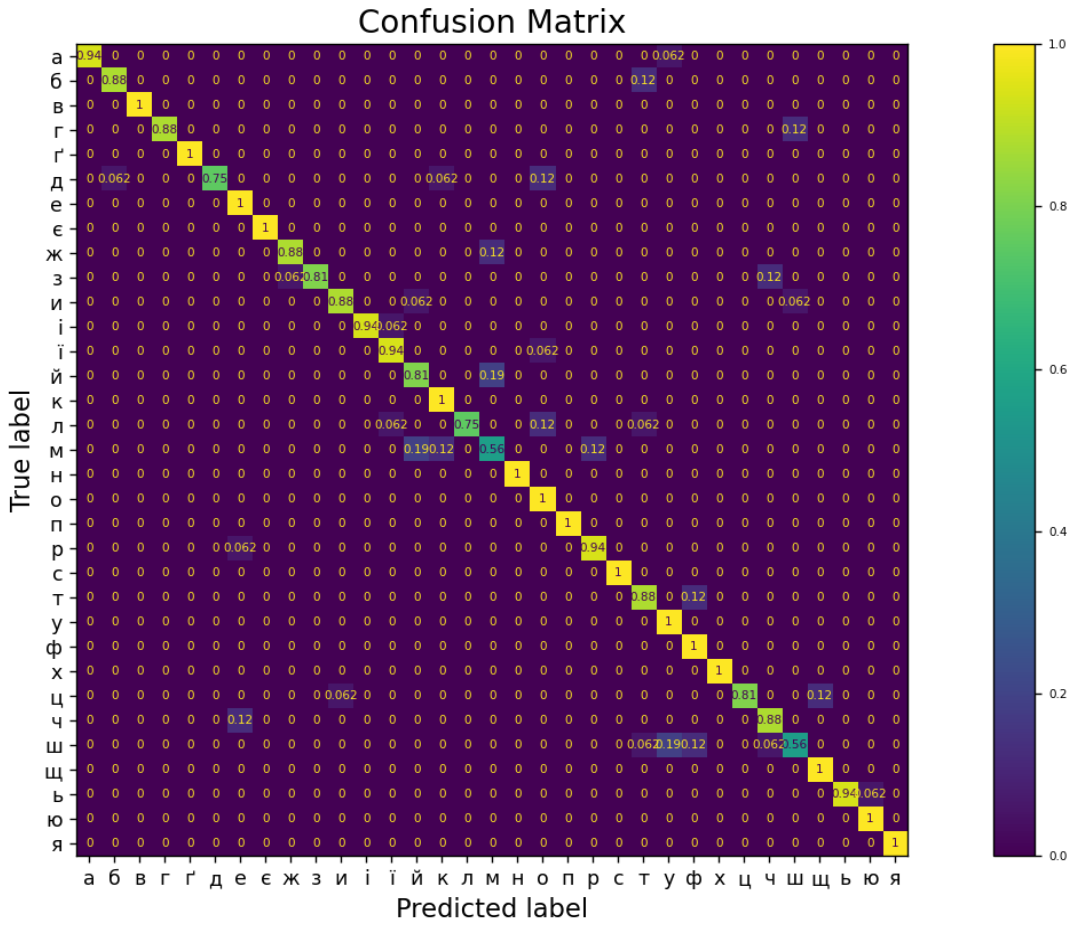


Рис. 10: матриця невідповідностей X3D-M на тестовій вибірці

Таблиця 3: звіт класифікації X3D-M на тестовій вибірці

Class	Precision	Recall	F1-score	Support
а	1.00	0.94	0.97	16
б	0.93	0.88	0.90	16
в	1.00	1.00	1.00	16

г	1.00	0.88	0.93	16
ґ	1.00	1.00	1.00	16
д	1.00	0.75	0.86	16
е	0.84	1.00	0.91	16
є	1.00	1.00	1.00	16
ж	0.93	0.88	0.90	16
з	1.00	0.81	0.90	16
и	0.93	0.88	0.90	16
і	1.00	0.94	0.97	16
ї	0.88	0.94	0.91	16
й	0.76	0.81	0.79	16
к	0.84	1.00	0.91	16
л	1.00	0.75	0.86	16
м	0.64	0.56	0.60	16
н	1.00	1.00	1.00	16
о	0.76	1.00	0.86	16
п	1.00	1.00	1.00	16
р	0.88	0.94	0.91	16
с	1.00	1.00	1.00	16
т	0.78	0.88	0.82	16
у	0.80	1.00	0.89	16
ф	0.80	1.00	0.89	16
х	1.00	1.00	1.00	16
ц	1.00	0.81	0.90	16
ч	0.82	0.88	0.85	16
ш	0.75	0.56	0.64	16

щ	0.89	1.00	0.94	16
ь	1.00	0.94	0.97	16
ю	0.94	1.00	0.97	16
я	1.00	1.00	1.00	16

accuracy	-	-	0.91	528
macro avg	0.92	0.91	0.91	528
weighted avg	0.92	0.91	0.91	528

Отримана система демонструє здатність розпізнавати окремі жести в умовах, далеких від контрольованих: за наявності різних рук, фонів, освітлення та положень у кадрі. Загальна точність top-1 становить 91%, що узгоджується як із макросередніми, так і зі зваженими значеннями F1-метрики.

Аналіз і обмеження. З одного боку, орієнтуючись на усереднені показники та звертаючись до прикладів тестових даних (рис. 10), можна говорити про досить високий рівень узагальнення моделі. Водночас детальні результати (табл. 3) свідчать про значну нерівномірність: поряд із ідеальними передбаченнями для окремих класів (наприклад, «в», «ґ», «є», «х», «я» із F1 = 100%) спостерігаються суттєво слабші результати для інших (зокрема «м» та «ш», де F1 становить 60% і 64% відповідно).

Для глибшого розуміння було додатково проаналізовано матрицю помилок на тренувальній вибірці (рис. 11):

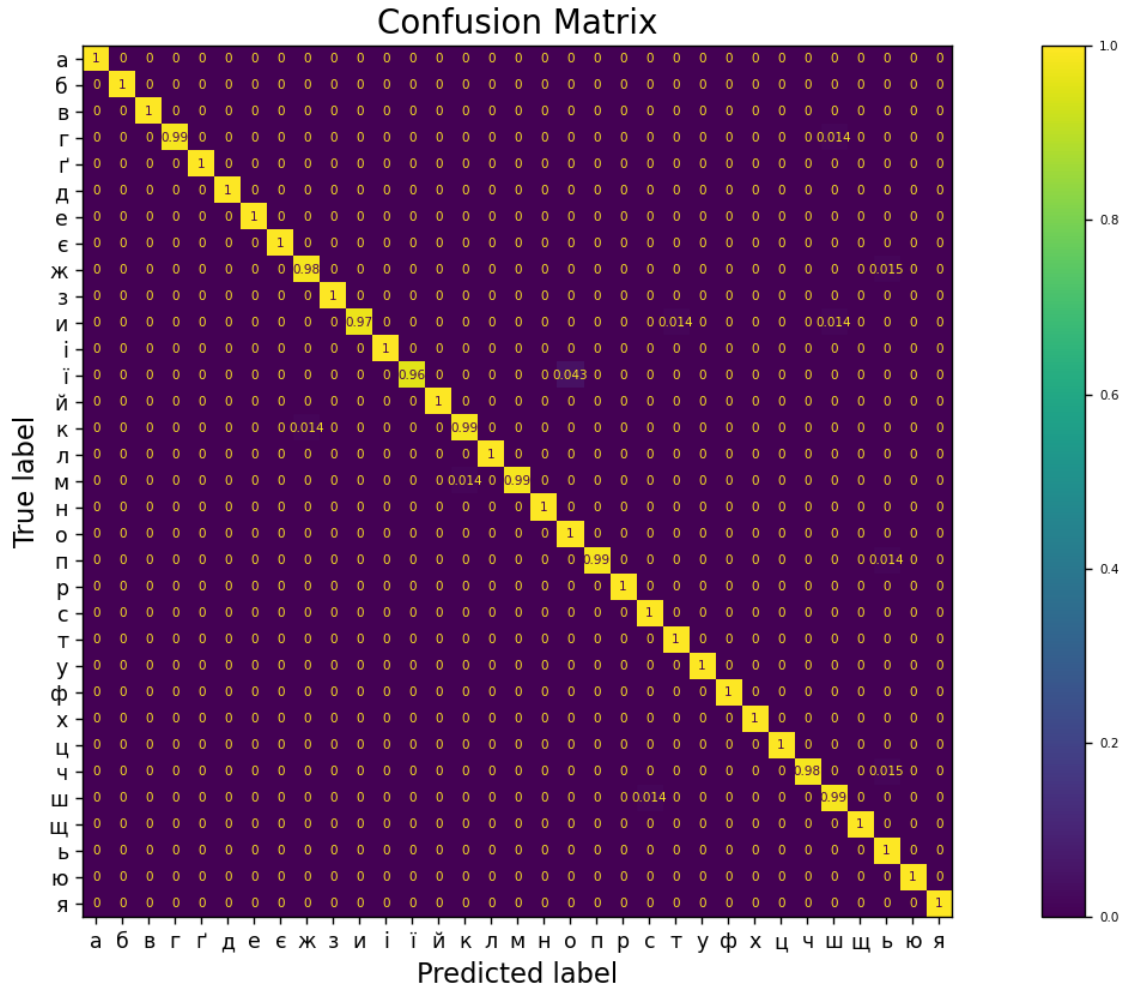


Рис. 11: матриця невідповідностей X3D-M на тренувальній вибірці

У ній більшість значень наближені до одиниці, а також відсутня помітна відповідність між помилками на тренувальних і тестових даних. Більше того, дослідження тестової матриці (рис. 10) показує, що модель, наприклад, плутає літеру «м» із «й», «к», «р», хоча ці жести суттєво відрізняються за конфігурацією пальців. Аналогічна ситуація спостерігається для «ш», яку модель іноді класифікує як «ч», «ф» або «у», попри явні відмінності між ними.

Ці спостереження дають підстави припустити, що навчальна вибірка, окрім корисних закономірностей, містить і небажані кореляції між класами, які модель використовує через їхню простішу відтворюваність. Відповідно, можна зробити висновок про недостатній обсяг даних: їх поки що не вистачає для адекватного

наближення цільового розподілу. Додатково слід врахувати обмеженість валідаційної та тестової підмножин — по 16 прикладів на клас явно недостатньо для повноцінної перевірки узагальнювальної здатності, тому реальна кількість слабких місць моделі може бути більшою, ніж зафіксовано.

3.2. Результати тренування ЗНМ для двоетапної класифікації через Mediapipe Hand Landmarker

Для оцінювання ефективності комбінації Mediapipe Hand Landmarker та класифікатора ключових точок (надалі «Mediapipe + GC») було побудовано матрицю невідповідностей (рис. 12) та обчислено покласові значення метрик precision/recall (табл. 4) на тестовому сеті:

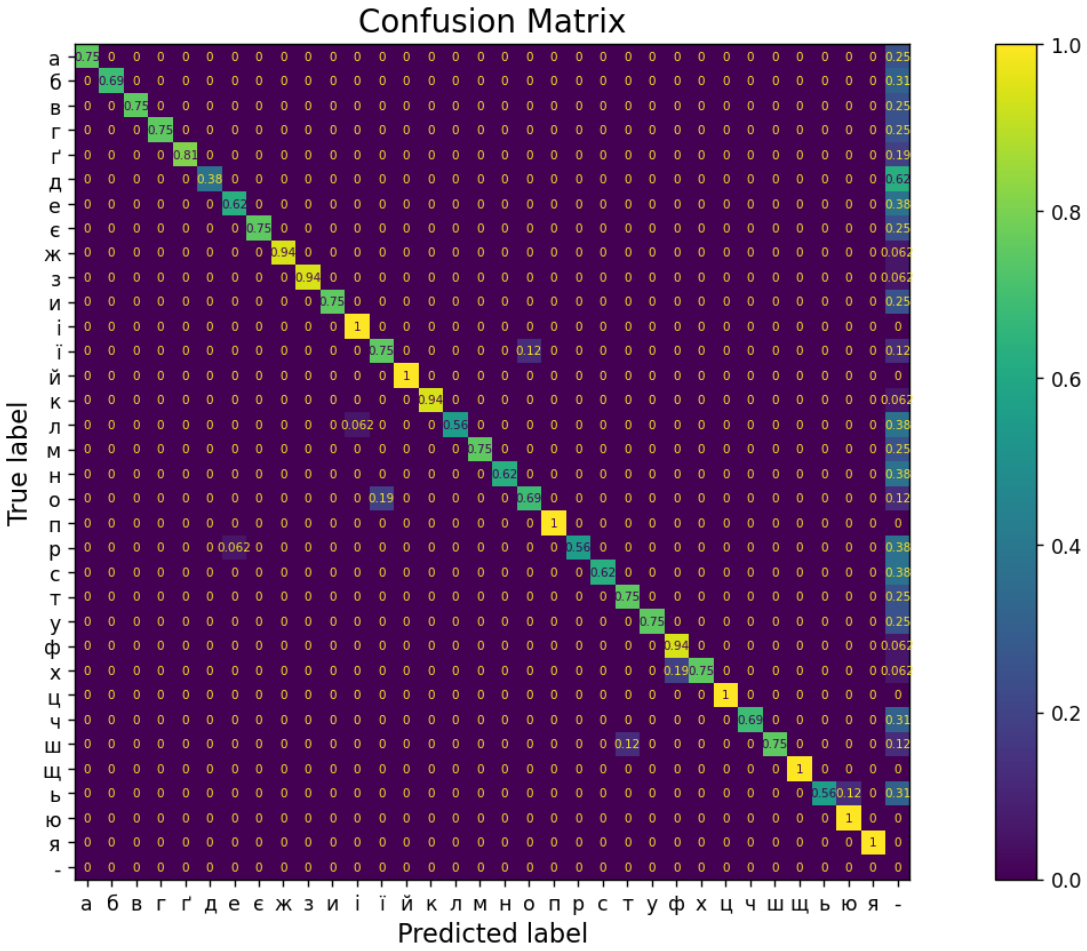


Рис. 12: матриця невідповідностей Mediapipe + GC на тестовій вибірці

Таблиця 4: звіт класифікації Mediapipe + GC на тестовій вибірці

class	*precision*	*recall*	*f1-score*	*support*
а	*1.00*	*0.75*	*0.86*	*16*
б	*1.00*	*0.69*	*0.81*	*16*
в	*1.00*	*0.75*	*0.86*	*16*
г	*1.00*	*0.75*	*0.86*	*16*
ґ	*1.00*	*0.81*	*0.90*	*16*
д	*1.00*	*0.38*	*0.55*	*16*
е	*0.91*	*0.62*	*0.74*	*16*
є	*1.00*	*0.75*	*0.86*	*16*
ж	*1.00*	*0.94*	*0.97*	*16*
з	*1.00*	*0.94*	*0.97*	*16*
и	*1.00*	*0.75*	*0.86*	*16*
і	*0.94*	*1.00*	*0.97*	*16*
ї	*0.80*	*0.75*	*0.77*	*16*
й	*1.00*	*1.00*	*1.00*	*16*
к	*1.00*	*0.94*	*0.97*	*16*
л	*1.00*	*0.56*	*0.72*	*16*
м	*1.00*	*0.75*	*0.86*	*16*
н	*1.00*	*0.62*	*0.77*	*16*
о	*0.85*	*0.69*	*0.76*	*16*
п	*1.00*	*1.00*	*1.00*	*16*
р	*1.00*	*0.56*	*0.72*	*16*
с	*1.00*	*0.62*	*0.77*	*16*
т	*0.86*	*0.75*	*0.80*	*16*
у	*1.00*	*0.75*	*0.86*	*16*

ф	*0.83*	*0.94*	*0.88*	*16*
х	*1.00*	*0.75*	*0.86*	*16*
ц	*1.00*	*1.00*	*1.00*	*16*
ч	*1.00*	*0.69*	*0.81*	*16*
ш	*1.00*	*0.75*	*0.86*	*16*
щ	*1.00*	*1.00*	*1.00*	*16*
ь	*1.00*	*0.56*	*0.72*	*16*
ю	*0.89*	*1.00*	*0.94*	*16*
я	*1.00*	*1.00*	*1.00*	*16*
-	*0.00*	*0.00*	*0.00*	*0*
accuracy	*-*	*-*	*0.78*	*528*
macro avg	*0.94*	*0.76*	*0.83*	*528*
weighted avg	*0.97*	*0.78*	*0.86*	*528*

Додаткові статистики включають аналіз помилок при роботі (рис. 13):

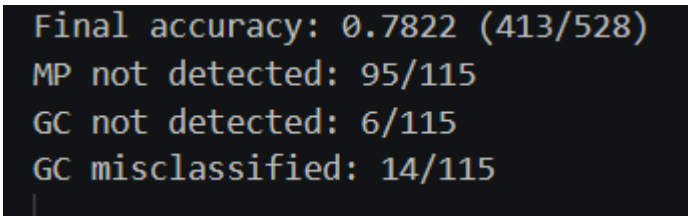


Рис. 13: деталі помилок, зроблених багатоетапною моделлю

З рисунку слідує те що із 528 жестів не було розпізнано 115, із котрих 95 не розпізнав mediapipe, класифікатор не розпізнав 6, та помилився на 14-ти. Також варто відмітити параметри top-1 accuracy = 78%, top-1 F1-міра = 86% (інформативним є показник weighted avg, оскільки macro avg помилково враховує штучно створений клас «-» (немає руки), додаючи зайві занулені показники).

Аналіз та обмеження. Головною проблемою розпізнавання дактилем української жестової мови, котрі собою являють динамічні варіації, залишається оклюзія та складність фонів. Із рисунку 4 слідує те, що абсолютна більшість помилок стосується не класифікатора, а моделі Mediapipe Hand Landmarker[21], котра не змогла розпізнати руки у складних середовищах — і це не дивно, оскільки остання оптимізована з нахилом на швидкодію, а не якість.

Щодо помилок класифікації — класифікатор ніяким чином, при будь-яких варіаціях тренувань, не зможе може уникнути помилок — оклюзія, тобто тремтіння ключових точок між фреймами відео, у багатьох випадках стиратиме критично важливу інформацію — плутати положення пальців.

З іншого боку, дана натренована багатоетапна система являє собою по-своєму стабільну відносно першої моделі варіацію, оскільки головні проблеми задачі, тобто візуальність — тіні, кольори рук, одяг, фон, тощо — перекладені на готову модель, котра натренована на великих (> 100 тисяч фото[21]) кількостях даних та має дуже і дуже гарну генералізацію.

Тобто, у рамках даного набору тестувальних даних ця модель отримала лише 78% accuracy (збалансований сет) — практичної якості роботи. Але — сам набір тестувальних даних за початковими умовами створення не може бути вичерпним — у ньому, як і в тренувальному та валідаційному сетах, містяться лише білі руки, скінченна кількість фонів, обмежена кількість тіней. А тому питання повного порівняння якості результатів залишається відкритим.

ВИСНОВКИ

Дана робота детально описує спроби виконання задачі розпізнавання дактилем української жестової мови з ціллю здобути максимальну генералізацію за обмеженої кількості даних.

Було проведено два експерименти. Перший включає спосіб одноетапної обробки вхідного зображення, використовує оптимізовану модель та навчання з перенесенням. Другий включає використання багатоетапної обробки зображення. Першим етапом є застосування уже натренованої доступної моделі (у даній праці у якості такої моделі було вибрано Mediapipe Hand Landmarker[21]) для отримання із зображення ключових точок цільової руки. Другим етапом є застосування натренованого на сконвертованих у ключові точки наявних даних класифікатора, котрий отримує послідовності координат ключових точок та класифікує жест за ними.

Деталі першого способу вирішення задачі продемонстровано у розділі 1. Пункт 1.1 докладно розписує особливості використаної архітектури та опорний датасет, на якому остання була претренована. У пункті 1.2 розписано та продемонстровано візуальну різноманітність записаної вибірки [17], метадані — збалансованість, розміри, розподіл сетів, особливості запису тощо — та деталі до кожного із сетів.

Пункт 1.3 покриває аугментацію даних, котра застосована для збільшення візуальної різноманітності: випадкове обрізання з масштабуванням, зміни яскравості, насичення, відтінку, випадкове віддзеркалення руки (перетворення з правої на ліву), повороти у випадкову сторону з випадковою мірою, різноманітні зашумлення.

Пункт 1.4 концентрує увагу на нюансах навчання за умови незбалансованості записаної вибірки — для уникнення зміщення моделі у бік розповсюдженіших класів при тренуваннях застосовано імовірносний відбір пакетів на основі обернених частоті класів вагових коефіцієнтів. Також вирівняний розподіл за необхідності додатково масштабується за результатами аналізу наявних помилок моделі, із використанням

логарифмічного оберненого зважування точності по діагоналі матриці невідповідностей.

Пункт 1.5 фокусується на використаних для тренування градієнтному оптимізаторі та критерію втрат. У якості критерію було використано класичну, практикою перевірену перехресну ентропію. У якості градієнтного оптимізатора використано AdamW[] — версія, котра виносить затухання ваг як окремий доданок при обчисленні градієнтів, запобігаючи паплюженню моментів, притаманному орігінальному Adam[].

Пункт 1.6 розписує використаний нестандартний планувальник швидкості навчання. За основу планувальника взято Cosine Annealing with Warm Restarts[20], але додатково на кожному циклі вхідна базова швидкість навчання масштабується за множником згасання — це дозволяє регулювати «міру розбіжності» моделі на межах циклів.

Пункт 1.7 розписує решту параметрів, котрі не потребують детального опису, та інші нюанси роботи, зокрема пов’язані з продуктивністю при тренуванні.

Другий розділ описує, відповідно, багатоетапний спосіб вирішення задачі розпізнавання дактилем української жестової мови. Пункт 2.1 фокусується на базовому описі архітектури використаної натренованої моделі розпізнавання ключових точок Mediapipe Hand Landmarker[21].

Пункт 2.2, натомість, містить опис архітектури класифікації послідовності ключових точок. Використано два темпоральні енкодери, один з яких працює з оригінальними координатами точок, а інший – з прискореннями між фреймами вхідного відео. Результати роботи цих енкодерів агрегуються та передаються у спільний класифікатор для фінального аналізу.

Пункт 2.3 фокусується на нюансах корвертування описаної у пункті 1.2 вибірки до простору ключових точок — детально розбирає що і як наскільки якісно було сконвертовано, що вдалося відновити інтерполяцією а що було повністю втрачено при

конвертації, оскільки Mediapipe Hand Landmarker не зміг побачити руку у тій чи іншій відео-сцені. Середній показник втрат при конвертації склав 3.229% для тренувального сету, 3.409% даних для валідаційного. Тестувальний сет конвертовано не було — тестування проводилося за повним циклом роботи.

У пункті 2.4 детально розписано застосовані аугментації для ключових точок — випадкове розтягування руки по ширині та довжині, віддзеркалення поворот у сторону та випадкове тремтіння ключових точок. Пункт 2.5 описує застосовані ваги коефіцієнтів для балансування із конфігурацією, аналогічно пункту 1.4.

Пункт 2.6 описує застосовані для тренування оптимізатор, критерій, та планувальник — котрі є тими ж, що використані для першого експерименту та зазначені у пунктах 1.5 й 1.6. А пункт 2.7 описує інші гіперпараметри, котрі не потребують роз’яснення нюансів.

Третій розділ описує результати тренування та тестування, отримані для кожного з експериментів, та детальний їх аналіз, включаючи побудову матриць плутаниць, покласового класифікаційного звіту та обробку зображених на них значень.

У першому пункті було розписано результати роботи способу одноетапної класифікації — Отримані top-1 accuracy та f1-score приблизно рівні 91% та 91% відповідно.

У другому пункті було показано практичні результати багатоетапної класифікації — Отримані top-1 accuracy та f1-score приблизно рівні 78% та 86% відповідно.

З одного боку, чітко видно що другий спосіб вирішення задачі показав значно гірші результати, ніж перший. З іншого ж боку, головний критерій якості моделі все-таки позначає не роботу на тому чи іншому датасеті, а спроможність у реальному світі — І у цьому випадку другий спосіб може не поступатися першому. Причиною цього припущення є недостатня різноманітність записаного набору, включаючи як

тренувальну вибірку, на котрій спосіб 1 був дотренований, так і тестувальну — усього по 16 зразків на кожен із 33 класів. Точніше кажучи, у наборі даних містяться виключно руки українців, тобто немає збалансованості у плані рас, кольору шкіри та інших потенційних анатомічних особливостей. А тому при реальній роботі результати можуть змінитися місцями, оскільки MediaPipe Hand Landmarker натренований на значно різноманітніших даних.

Водночас потенціал способу 1 не повинен поступатися такому для способу 2 за умови достатньої кількості даних, але обговорення «за достатньої кількості даних» не є ціллю цієї роботи.

Подальші дослідження фокусуватимуться на поліпшенні аугментацій, особливо у темпоральному вимірі, та способах отримання нових, включаючи синтетичних, даних з ціллю досягнення високої різноманітності наборів.

Коди створених у рамках праці програм знаходяться у відкритому доступі на репозиторіях GitHub. Деталі описано у додатку Г.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Prasuhn L., Oyamada Y., Mochizuki Y., Ishikawa H. A hog-based hand gesture recognition system on a mobile device. IEEE International Conference on Image Processing (ICIP). 2014. P. 3973–3977. https://doi.org/10.1109/ICIP.2014.7025807
2. Dardas N. H., Georganas N. D. Real-time hand gesture detection and recognition using bag-of-features and support vector machine techniques. IEEE Transactions on Instrumentation and Measurement. 2011. Vol. 60(11), 3592–3607. https://doi.org/10.1109/TIM.2011.2161140
3. Krak I., Kudin G., Kulias A. Multidimensional scaling by means of pseudoinverse operations. Cybernetics and Systems Analysis. 2019. Vol. 55(1), 22–29. https://doi.org/10.1007/s10559-019-00108-9
4. Li D., Rodriguez Opazo C., Yu X., Li H. Word-level deep sign language recognition from video: a new large-scale dataset and methods comparison. IEEE Winter Conference on Applications of Computer Vision (WACV). 2020. P. 1459–1469. https://arxiv.org/abs/1910.11006
5. Vaezi Joze H. R., Koller O. MS-ASL: A large-scale data set and benchmark for understanding American Sign Language. BMVC (British Machine Vision Conference). 2019. https://arxiv.org/abs/1812.01053
6. Duarte A., Palaskar S., Ventura L., Ghadiyaram D., DeHaan K., Metze F., Torres J., Giró-i-Nieto X. How2Sign: a large-scale multimodal dataset for continuous American sign language. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2021. Vol. –, P. 2734-2743. https://doi.org/10.1109/CVPR46437.2021.00276
7. Sincan O. M., Keles H. Y. AUTSL: A large-scale multi-modal Turkish Sign Language Dataset and baseline methods. IEEE Access. 2020. Vol. 8, 181340−181355. https://doi.org/10.1109/ACCESS.2020.3028072
8. Zhang J., Zhou W., Xie C., Pu J., Li H. Chinese sign language recognition with adaptive HMM. IEEE International Conference on Multimedia & Expo (ICME). 2016. P. 1–6.
9. Kondratiuk S., Krak I., Kuznetsov V., Kulias A. Using the temporal data and three-dimensional convolutions for sign language alphabet recognition. Conference: CMIS 2022. 2022. Vol. –, P. 78–87. https://doi.org/10.32782/cmis/3137-7
10. Howard A., Sandler M., Chu G., Chen L.-C., Chen B., Tan M., Wang W., Zhu Y., Pang R., Vasudevan V., Le Q. V., Adam H. Searching for MobileNetV3. IEEE/CVF International Conference on Computer Vision (ICCV). 2019. P. 1314–1324. https://doi.org/10.1109/ICCV.2019.00140
11. Feichtenhofer C. X3D: Expanding architectures for efficient video recognition. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020. P. 203-213. https://doi.org/10.1109/CVPR42600.2020.00028

12. Carreira J., Zisserman A. Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. arXiv preprint arXiv:1705.07750. 2017. https://arxiv.org/abs/1705.07750
13. Kay W., Carreira J., Simonyan K., Zhang B., Hillier C., Vijayanarasimhan S., Viola F., Green T., Back T., Natsev P., Suleyman M., Zisserman A. The Kinetics Human Action Video Dataset. 2017. https://arxiv.org/abs/1705.06950
14. Carreira J., Noland E., Banki-Horváth Á., Hillier C., Zisserman A. A short note about Kinetics-600. 2018. https://arxiv.org/abs/1808.01340
15. Sigurdsson G. A., Varol G., Wang X., Farhadi A., Laptev I., Gupta A. Hollywood in Homes: Crowdsourcing Data Collection for Activity Understanding. European Conference on Computer Vision (ECCV). 2016.https://arxiv.org/abs/1604.01753
16. Gu C., Sun C., Ross D. A., Vondrick C., Pantofaru C., Li Y., Vijayanarasimhan S., Toderici G., Ricco S., Sukthankar R., Schmid C., Malik J. AVA: A video dataset of spatio-temporally localized atomic visual actions. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2018. https://arxiv.org/abs/1705.08421
17. Krak I., Kondratiuk S., Kuznetsov V., Yefremov N., Zadorozhnyi M. A multimodal corpus of Ukrainian dactyls (Videos and Frames). Zenodo. Nov 15, 2025. https://doi.org/10.5281/zenodo.17618406
18. Kingma D. P., Ba J. Adam: A method for stochastic optimization. ICLR (Poster). 2015. https://arxiv.org/abs/1412.6980
19. Loshchilov I., Hutter F. Decoupled weight decay regularization. International Conference on Learning Representations (ICLR). 2019.https://arxiv.org/abs/1711.05101
20. Loshchilov I., Hutter F. SGDR: Stochastic Gradient Descent with Warm Restarts. 2016. https://arxiv.org/abs/1608.03983
21. Zhang F., Bazarevsky V., Vakunov A., Tkachenka A., Sung G., Chang C.-L., Grundmann M. MediaPipe Hands: On-device real-time hand tracking. CV4ARVR Workshop. 2020. https://arxiv.org/abs/2006.10214

ДОДАТОК А: код екстрактора кадрів із відео

```
import cv2
import os
from PIL import Image
import shutil

#========== GLOBAL SETTINGS ==========
frame_size_needed = (224,224)
max_w, max_h = 1200, 700 #for display of cv2, needed for correct cropping

# folder with videos to process, presumably just camera folder
camera_folder = "../Video"

# folder to put frames into after extraction
dataset_folder = "../data/gg"

# where to put vids into after the job`s done, useful for preventing confusions
processed_folder = "../Video/processed_shit"


def extract_frames(video_path, step=-1, warmup=0, fr_retrieve = 16):

    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print(f"Error: Could not open video: {video_path}")
        return

    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    if step == -1:
        step = frame_count // fr_retrieve
    frames_to_return = []
    i = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if i >= warmup and i % step == 0 and len(frames_to_return) < fr_retrieve:
            frames_to_return.append(frame)
        i += 1

    return frames_to_return

def enable_crop(input_frames):
    drawing = False
```

```
ix, iy = -1, -1
crop_rect = []

#========== SELECT CROP ==========
def draw_square(event, x, y, flags, param):
    nonlocal ix, iy, drawing, crop_rect

    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        ix, iy = x, y

    elif event == cv2.EVENT_MOUSEMOVE and drawing:
        img_copy = img_resized.copy()

        dx = x - ix
        dy = y - iy

        # Determine side length for square
        side = min(abs(dx), abs(dy))

        # Preserve drag direction (corner)
        if dx >= 0 and dy >= 0:
            x2, y2 = ix + side, iy + side
        elif dx < 0 and dy >= 0:
            x2, y2 = ix - side, iy + side
        elif dx >= 0 and dy < 0:
            x2, y2 = ix + side, iy - side
        else:  # dx < 0 and dy < 0
            x2, y2 = ix - side, iy - side

        cv2.rectangle(img_copy, (ix, iy), (x2, y2), (0, 255, 0), 2)
        cv2.imshow(window_name, img_copy)

    elif event == cv2.EVENT_LBUTTONUP:
        drawing = False

        dx = x - ix
        dy = y - iy
        side = min(abs(dx), abs(dy))

        if dx >= 0 and dy >= 0:
            x2, y2 = ix + side, iy + side
        elif dx < 0 and dy >= 0:
            x2, y2 = ix - side, iy + side
        elif dx >= 0 and dy < 0:
            x2, y2 = ix + side, iy - side
        else:
```

```
                x2, y2 = ix - side, iy - side

            crop_rect[:] = [min(ix, x2), min(iy, y2), side, side]


    if len(input_frames) == 0:
        print("Could not read first frame - there is none.")
        return input_frames

    sample_img = input_frames[0]
    window_name = f"Select the rectangle you want your frames to be cropped to:"
    cv2.namedWindow(window_name)
    cv2.setMouseCallback(window_name, draw_square)

    h, w = sample_img.shape[:2]
    scale = 1 # will be changed if img needs resizing

    if w > max_w or h > max_h:
        img_cpy = sample_img.copy()
        scale = min(max_w / w, max_h / h)
        new_w = int(w * scale)
        new_h = int(h * scale)
        img_resized = cv2.resize(img_cpy, (new_w, new_h), interpolation=cv2.INTER_AREA)
    else:
        img_resized = sample_img

    cv2.imshow(window_name, img_resized)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    if not crop_rect or crop_rect[-1] == 0 or crop_rect[-2] == 0:
        print("No crop selected.")
        return input_frames

    # ========== ENABLE CROP PER-FRAME ==========
    x, y, w, h = [round(v / scale) for v in crop_rect]


    cropped = [frame[y:y + h, x:x + w] for frame in input_frames]
    return cropped

def tag_as_processed(input_vid_path):
    global processed_folder
    os.makedirs(processed_folder, exist_ok=True)

    if not hasattr(tag_as_processed, "scene_dir"):
        scenes = [scene_dir for scene_dir in os.listdir(processed_folder)
```

```
                if os.path.isdir(os.path.join(processed_folder, scene_dir))]
        spc = len(scenes) #scenes_processed_count
        tag_as_processed.scene_dir = os.path.join(processed_folder, f"scene{spc + 1}")

    os.makedirs(tag_as_processed.scene_dir, exist_ok=True)
    shutil.move(src=input_vid_path, dst=os.path.join(tag_as_processed.scene_dir,
os.path.basename(input_vid_path)))

def read_camera_to_dataset(output_data_root_dir):
    global camera_folder

    vid_names = sorted([v_name for v_name in os.listdir(camera_folder) if
                   not os.path.isdir(os.path.join(camera_folder, v_name))
                   and ".ini" not in v_name]) #remove desktop.ini sneaking in


    #highly configurable: any letter sequence can be entered
    # just be sure your videos in camera_folder match EXACTLY (order matters).
    class_names = [
        'а', 'б', 'в', 'г', 'ґ', 'д', 'е', 'є',
        'ж', 'з', 'и', 'і', 'ї', 'й', 'к', 'л',
        'м', 'н', 'о', 'п', 'р', 'с', 'т', 'у',
        'ф', 'х', 'ц', 'ч', 'ш', 'щ', 'ь', 'ю', 'я'
    ]

    if len(vid_names) != len(class_names):
        print(f"gotta need {len(class_names)} gesture vids. got: {len(vid_names)}")
        return

    for class_idx in range(len(class_names)):
        class_path = os.path.join(output_data_root_dir, class_names[class_idx])
        os.makedirs(class_path, exist_ok=True)

        vid_dirs_existed = sorted([d for d in os.listdir(class_path)
                                   if os.path.isdir(os.path.join(class_path, d))])

        #creating new folder - a bit complicated to be sure it wont overwrite smth.
        vids_count = len(vid_dirs_existed)
        while True:
            output_vid_path = os.path.join(class_path, f"vid{vids_count + 1}_test")
            if not os.path.exists(output_vid_path):
                break
            vids_count += 1
        os.makedirs(output_vid_path, exist_ok=False)


        input_vid_path = os.path.join(camera_folder, vid_names[class_idx])
```

```
        print(f"video '{class_names[class_idx]}': extracting frames...")
        frames = extract_frames(video_path=input_vid_path) #gotta get 16 frames
        print(f"video '{class_names[class_idx]}': enabling crop...")
        frames = enable_crop(input_frames=frames)
        frames_rgb = [cv2.cvtColor(f, cv2.COLOR_BGR2RGB) for f in frames]

        print(f"video '{class_names[class_idx]}': saving...")
        for i, f in enumerate(frames_rgb):
            img = Image.fromarray(f)
            img = img.resize(size=frame_size_needed)
            img_path = os.path.join(output_vid_path, f"frame_{(i+1):05d}.jpg")
            if not os.path.exists(img_path):
                img.save(img_path, format="JPEG")
            else:
                print(f"video '{class_names[class_idx]}': folder_already_exists.\n")
                break

        tag_as_processed(input_vid_path=input_vid_path)
        print(f"video '{class_names[class_idx]}': done.\n")


if __name__ == "__main__":
    read_camera_to_dataset(dataset_folder)
```

ДОДАТОК Б: деталі архітектури для класифікації

Деталі архітектури можна побачити на прикладі даних, отриманих за допомогою torchinfo.summary(). На прикладі вхідного батчу розміром 64, вона повністю зібрала інформацію про кожен шар архітектури, включаючи форми вхідних і вихідних тензорів та кількість параметрів. Після таблиці наведена загальна кількість параметрів моделі та оцінки щодо розмірів з точки зору пам’яті.

Layer (type (var_name):depth-idx)	Input Shape	Output Shape	Param #
GestureNetV1 (GestureNetV1)	[64, 16, 42]	[64, 33]	--
├─HandNormalizer (normalizer): 1-1	[64, 16, 42]	[64, 16, 42]	--
├─TemporalEncoder (pos_encoder): 1-2	[64, 16, 42]	[64, 128]	--
│ └─Sequential (stem): 2-1	[64, 42, 16]	[64, 96, 16]	--
│ │ └─Conv1d (0): 3-1	[64, 42, 16]	[64, 96, 16]	12,192
│ │ └─BatchNorm1d (1): 3-2	[64, 96, 16]	[64, 96, 16]	192
│ │ └─ReLU (2): 3-3	[64, 96, 16]	[64, 96, 16]	--
│ └─ResidualBlock (layer1): 2-2	[64, 96, 16]	[64, 128, 16]	--
│ │ └─Conv1d (conv1): 3-4	[64, 96, 16]	[64, 64, 16]	18,432

│ │ │ └─BatchNorm1d (bn1): 3-5	[64, 64, 16]	[64, 64, 16]	128
│ │ │ └─Conv1d (conv2): 3-6	[64, 64, 16]	[64, 128, 16]	24,576
│ │ │ └─BatchNorm1d (bn2): 3-7	[64, 128, 16]	[64, 128, 16]	256
│ │ │ └─Sequential (shortcut): 3-8	[64, 96, 16]	[64, 128, 16]	--
│ │ │ │ └─Conv1d (0): 4-1	[64, 96, 16]	[64, 128, 16]	12,288
│ │ │ │ └─BatchNorm1d (1): 4-2	[64, 128, 16]	[64, 128, 16]	256
│ │ └─ResidualBlock (layer2): 2-3	[64, 128, 16]	[64, 128, 8]	--
│ │ │ └─Conv1d (conv1): 3-9	[64, 128, 16]	[64, 64, 8]	24,576
│ │ │ └─BatchNorm1d (bn1): 3-10	[64, 64, 8]	[64, 64, 8]	128
│ │ │ └─Conv1d (conv2): 3-11	[64, 64, 8]	[64, 128, 8]	24,576
│ │ │ └─BatchNorm1d (bn2): 3-12	[64, 128, 8]	[64, 128, 8]	256
│ │ │ └─Sequential (shortcut): 3-13	[64, 128, 16]	[64, 128, 8]	--
│ │ │ │ └─Conv1d (0): 4-3	[64, 128, 16]	[64, 128, 8]	16,384
│ │ │ │ └─BatchNorm1d (1): 4-4	[64, 128, 8]	[64, 128, 8]	256
│ │ └─AdaptiveAvgPool1d (avg_pool): 2-4	[64, 128, 8]	[64, 128, 1]	--
│ ├─TemporalEncoder (vel_encoder): 1-3	[64, 16, 42]	[64, 128]	--
│ │ └─Sequential (stem): 2-5	[64, 42, 16]	[64, 96, 16]	--
│ │ │ └─Conv1d (0): 3-14	[64, 42, 16]	[64, 96, 16]	12,192
│ │ │ └─BatchNorm1d (1): 3-15	[64, 96, 16]	[64, 96, 16]	192
│ │ │ └─ReLU (2): 3-16	[64, 96, 16]	[64, 96, 16]	--
│ │ └─ResidualBlock (layer1): 2-6	[64, 96, 16]	[64, 128, 16]	--
│ │ │ └─Conv1d (conv1): 3-17	[64, 96, 16]	[64, 64, 16]	18,432
│ │ │ └─BatchNorm1d (bn1): 3-18	[64, 64, 16]	[64, 64, 16]	128
│ │ │ └─Conv1d (conv2): 3-19	[64, 64, 16]	[64, 128, 16]	24,576
│ │ │ └─BatchNorm1d (bn2): 3-20	[64, 128, 16]	[64, 128, 16]	256
│ │ │ └─Sequential (shortcut): 3-21	[64, 96, 16]	[64, 128, 16]	--
│ │ │ │ └─Conv1d (0): 4-5	[64, 96, 16]	[64, 128, 16]	12,288
│ │ │ │ └─BatchNorm1d (1): 4-6	[64, 128, 16]	[64, 128, 16]	256
│ │ └─ResidualBlock (layer2): 2-7	[64, 128, 16]	[64, 128, 8]	--
│ │ │ └─Conv1d (conv1): 3-22	[64, 128, 16]	[64, 64, 8]	24,576
│ │ │ └─BatchNorm1d (bn1): 3-23	[64, 64, 8]	[64, 64, 8]	128
│ │ │ └─Conv1d (conv2): 3-24	[64, 64, 8]	[64, 128, 8]	24,576
│ │ │ └─BatchNorm1d (bn2): 3-25	[64, 128, 8]	[64, 128, 8]	256
│ │ │ └─Sequential (shortcut): 3-26	[64, 128, 16]	[64, 128, 8]	--
│ │ │ │ └─Conv1d (0): 4-7	[64, 128, 16]	[64, 128, 8]	16,384
│ │ │ │ └─BatchNorm1d (1): 4-8	[64, 128, 8]	[64, 128, 8]	256
│ │ └─AdaptiveAvgPool1d (avg_pool): 2-8	[64, 128, 8]	[64, 128, 1]	--
│ ├─Sequential (classifier): 1-4	[64, 256]	[64, 33]	--
│ │ └─Linear (0): 2-9	[64, 256]	[64, 256]	65,792
│ │ └─BatchNorm1d (1): 2-10	[64, 256]	[64, 256]	512

│ └─ReLU (2): 2-11	[64, 256]	[64, 256]	--
│ └─Dropout (3): 2-12	[64, 256]	[64, 256]	--
│ └─Linear (4): 2-13	[64, 256]	[64, 33]	8,481

Total params: 343,777

Trainable params: 343,777

Non-trainable params: 0

Total mult-adds (Units.MEGABYTES): 210.30

Input size (MB): 0.17

Forward/backward pass size (MB): 19.15

Params size (MB): 1.38

Estimated Total Size (MB): 20.70

ДОДАТОК В: код конвертації датасету в ключові точки

```
import mediapipe as mp
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
from mediapipe.tasks.python.vision import drawing_styles as mp_drawing_styles
from mediapipe.tasks.python.vision import drawing_utils as mp_drawing
from mediapipe.tasks.python.vision import HandLandmarksConnections as mp_hands
from mediapipe.tasks.python.components.containers.landmark import NormalizedLandmark

import numpy as np
import cv2
from scipy.interpolate import interp1d
from pathlib import Path
from PIL import Image
import logging
import matplotlib.pyplot as plt
import matplotlib
from copy import deepcopy
import mlflow

FPS=30
VID_LENGTH=16 #frames
MAX_MISSING_FRAMES=4
CONFIDENCE_TRESHOLD=0.7


def draw_landmarks_on_image(rgb_image, detection_result, raw_coords=False):
    margin = 10 # pixels
    if raw_coords:
        hand_landmarks_list = [build_landmark_list(detection_result)]
        #handedness_list = None
    else:
        hand_landmarks_list = detection_result.hand_landmarks
        #handedness_list = detection_result.handedness

    annotated_image = np.copy(rgb_image)

    # Loop through the detected hands to visualize.
    for idx in range(len(hand_landmarks_list)):
        hand_landmarks = hand_landmarks_list[idx]
        #handedness = handedness_list[idx] if handedness_list is not None else None

        # Draw the hand landmarks.
        mp_drawing.draw_landmarks(
        annotated_image,
        hand_landmarks,
```

```
            mp_hands.HAND_CONNECTIONS,
            mp_drawing_styles.get_default_hand_landmarks_style(),
            mp_drawing_styles.get_default_hand_connections_style())

            # Get the top left corner of the detected hand's bounding box.
            # height, width, _ = annotated_image.shape
            # x_coordinates = [landmark.x for landmark in hand_landmarks]
            # y_coordinates = [landmark.y for landmark in hand_landmarks]
            # text_x = int(min(x_coordinates) * width)
            # text_y = int(min(y_coordinates) * height) - margin

            # Draw handedness (left or right hand) on the image.
            # cv2.putText(annotated_image, f"{handedness[0].category_name}",
            #             (text_x, text_y), cv2.FONT_HERSHEY_DUPLEX,
            #             FONT_SIZE, HANDEDNESS_TEXT_COLOR, FONT_THICKNESS, cv2.LINE_AA)

        return annotated_image

    def build_landmark_list(raw_coords_2d_flattened):
        landmark_list = list()

        for x, y in raw_coords_2d_flattened.reshape(-1, 2):
            landmark = NormalizedLandmark()
            landmark.x = x
            landmark.y = y
            landmark.z = 0.0  # safe default
            landmark_list.append(landmark)

        return landmark_list


    def extract_keypoints(root_dir: Path, output_file: str, logging_file: str,
    interpolation_kind='cubic', ):
        logging.basicConfig(filename=logging_file, level=logging.INFO, encoding='utf-8')
        base_options = python.BaseOptions(model_asset_path='hand_landmarker.task')
        options = vision.HandLandmarkerOptions(base_options=base_options,
                                               running_mode=vision.RunningMode.VIDEO,
                                               min_hand_detection_confidence=0.75,
                                               min_hand_presence_confidence=0.5,
                                               min_tracking_confidence=0.75,
                                               num_hands=1)

        X = [] # Data
        y = [] # Labels
        failed_vids=[]
        interpolated_vids=[]
        classes = sorted([d.name for d in root_dir.iterdir() if d.is_dir()])
```

```
    for cls_idx, cls_name in enumerate(classes):
        class_dir = root_dir / cls_name

        for vid_dir in class_dir.iterdir():
            if not vid_dir.is_dir(): continue

            video_id = f'{cls_name}\\{vid_dir.name}'  # Unique ID for the video
            video_landmarks = [None]*VID_LENGTH
            valid_frames_mask = [False] * VID_LENGTH  # To track which frames are valid
            frame_imgs_bgr = [] #visualization

            detector = vision.HandLandmarker.create_from_options(options)


            frame_paths = sorted(list(vid_dir.glob('*.jpg')))[:VID_LENGTH]

            for frame_index, img_path in enumerate(frame_paths):
                frame_id = img_path.name
                frame = Image.open(img_path).convert('RGB')
                rgb_img = mp.Image(image_format=mp.ImageFormat.SRGB,
data=np.array(frame))

                timestamp_ms = int(frame_index * (1000 / FPS))
                results = detector.detect_for_video(rgb_img, timestamp_ms)

                frame_bgr=cv2.cvtColor(np.array(frame), cv2.COLOR_RGB2BGR)
                frame_imgs_bgr.append(frame_bgr)
                annotated_frame_bgr = draw_landmarks_on_image(frame_bgr, results)
                #cv2.imshow('MediaPipe Hands Recognition', annotated_frame_bgr)


                if results.hand_landmarks:
                    # Check confidence score of the first hand
                    hand_info = results.handedness[0][0]
                    conf = hand_info.score
                    label = hand_info.category_name
                    if conf > CONFIDENCE_TRESHOLD:  # Your adequacy threshold
                        frame_landmarks = np.array([[lm.x, lm.y] for lm in
results.hand_landmarks[0]]).flatten() #lm.z is not useful
                        video_landmarks[frame_index] = frame_landmarks
                        valid_frames_mask[frame_index] = True

                # if is_success:
                #     success_count += 1
                #     logging.info(f"☑[PASS] Video: \'{video_id}\' | Frame: {frame_id}
(Conf: {conf:.2f}, Label: {label})")
```

```
                # else:
                #     # Forward fill if possible, else zeros
                #     if len(video_landmarks) > 0 and np.any(video_landmarks[-1]):
                #         frame_landmarks = video_landmarks[-1]
                #         logging.warning(f"⚠️[HALF-FAIL] Video: \'{video_id}\' | Frame:
{frame_id} — Low Confidence/No Hand, forward-filled")

                #     else:
                #         logging.warning(f"❌[FAIL] Video: \'{video_id}\' | Frame:
{frame_id} — Low Confidence/No Hand")
                #         frame_landmarks = np.zeros(21*2)

                #     if video_id not in failed_vids:
                #         failed_vids.append(video_id)

            invalid_count = sum(not v for v in valid_frames_mask)
            success_count = VID_LENGTH — invalid_count

            if invalid_count == 0:
                X.append(video_landmarks)
                y.append(cls_idx)
                logging.info(f"--- ☑Video {video_id} Complete. Success Rate:
{success_count}/{VID_LENGTH} (interpolated {invalid_count} frames) ---")
                continue

            if invalid_count > MAX_MISSING_FRAMES:
                failed_vids.append(video_id)
                logging.info(f"---❌ Video {video_id}: skipped due to {invalid_count}
missing frames ---\n")
                continue

            # Interpolate missing frames, guaranteed to have at least 16-4=12 valid
frames for interpolation
            num_coords = 42  # 21 landmarks * 2 (x,y)
            interpolated_video_landmarks = deepcopy(video_landmarks)
            for coord_idx in range(num_coords):
                known_indices = [i for i, lm in enumerate(video_landmarks) if lm is not
None]
                known_values = [video_landmarks[i][coord_idx] for i in known_indices]

                interpolate_func = interp1d(x=known_indices, y=known_values,
kind=interpolation_kind,
                                            bounds_error=False, fill_value=(known_values[0],
known_values[-1])) #fill_value="extrapolate", too much outliers

                ready_vals = interpolate_func(range(VID_LENGTH))
```

```
                for i in range(VID_LENGTH):
                    if interpolated_video_landmarks[i] is None:
                        interpolated_video_landmarks[i] = np.zeros(num_coords)
                    interpolated_video_landmarks[i][coord_idx] = ready_vals[i]


            #log the interpolation to mlflow
            interpolated_vids.append(video_id)
            for i in range(VID_LENGTH):
                annotated_frame_bgr = draw_landmarks_on_image(frame_imgs_bgr[i],
interpolated_video_landmarks[i], raw_coords=True)
                annotated_frame_rgb = cv2.cvtColor(annotated_frame_bgr,
cv2.COLOR_BGR2RGB)
                if valid_frames_mask[i]:
                    mlflow.log_image(annotated_frame_rgb,
f"interpolated_vids/{vid_dir}/frame_{i}.jpg")
                else:
                    mlflow.log_image(annotated_frame_rgb,
f"interpolated_vids//{vid_dir}/frame_{i}_interpolated.jpg")


            # Now use interpolated_landmarks for X.append(...)
            logging.info(f"--- ☑Video {video_id} Complete. Success Rate:
{success_count}/{VID_LENGTH} (interpolated {invalid_count} frames) ---")
            X.append(interpolated_video_landmarks)
            y.append(cls_idx)


    logging.info(f"All videos done. failed videos:
{len(failed_vids)}/{len(X)+len(failed_vids)};\n")
    logging.info(f"All videos done. interpolated videos:
{len(interpolated_vids)}/{len(X)+len(failed_vids)};\n")

    mlflow.log_param('failed_videos', f"{len(failed_vids)}/{len(X)+len(failed_vids)}")
    mlflow.log_param('interpolated_videos',
f"{len(interpolated_vids)}/{len(X)+len(failed_vids)}")

    logging.info(f"Failed video names:")
    for i, vid_id in enumerate(failed_vids):
        logging.info(f'   {i}) \"{vid_id}\"')

    # Save as compressed numpy files
    np.savez_compressed(output_file, X=np.array(X, dtype=np.float32), y=np.array(y))
    mlflow.log_artifact(output_file, artifact_path=output_file)
    mlflow.log_artifact(logging_file, artifact_path=logging_file)
    print(f"Dataset saved to {output_file} with shape {np.array(X).shape}")
```

```
if __name__ == "__main__":
    # params = {
    #     'experiment_name': 'GC Keypoint Extraction',
    #     'run_name': 'keypoint_extraction_train',
    #     'root_dir': Path('data_images/train'),
    #     'output_file': 'data/data_train.npz',
    #     'logging_file': 'extraction_log_train.txt',
    #     "interpolation_kind": 'cubic'
    # }
    params = {
        'experiment_name': 'GC Keypoint Extraction',
        'run_name': 'keypoint_extraction_validation',
        'root_dir': Path('data_images/val'),
        'output_file': 'data/data_validation.npz',
        'logging_file': 'extraction_log_validation.txt',
        "interpolation_kind": 'cubic'
    }

    mlflow.set_tracking_uri("sqlite:///mlflow.db")
    mlflow.set_experiment(params['experiment_name'])


    with mlflow.start_run(run_name=params['run_name']):
        mlflow.log_params(params)
        extract_keypoints(params['root_dir'], params['output_file'],
params['interpolation_kind'])
```

ДОДАТОК Г: код створених у рамках роботи програм тренування

Повний код створених у рамках роботи тренувань розміщено у двох репозиторіях, оскільки ці праці відповідали двом окремим експериментам.

Перший експеримент, тобто одноетапна класифікація, розміщений за посиланням:

https://github.com/mihzad/GestureClassifier

Другий експеримент, тобто багатоетапна класифікація, розміщений за посиланням:

https://github.com/mihzad/KeypointGestureClassifier